

UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

PABLO DE PAULA CORREIA

**H2GÁS: UMA APLICAÇÃO WEB PARA GESTÃO DE ENTREGAS DE GÁS E
ÁGUA**

GUARAPUAVA

2025

PABLO DE PAULA CORREIA

H2GÁS: UMA APLICAÇÃO WEB PARA GESTÃO DE ENTREGAS DE GÁS E ÁGUA

H2Gas: Web Application for Gas and Water Delivery Management

Trabalho de Conclusão de Curso apresentado como requisito para obtenção do título de Tecnólogo em Tecnologia Em Sistemas Para Internet do Curso de Tecnologia Em Sistemas Para Internet da Universidade Tecnológica Federal do Paraná.

Orientador: Dr. Roni Fabio Banaszewski

GUARAPUAVA

2025



[4.0 Internacional](https://creativecommons.org/licenses/by/4.0/)

Esta licença permite compartilhamento, remixe, adaptação e criação a partir do trabalho, mesmo para fins comerciais, desde que sejam atribuídos créditos ao(s) autor(es). Conteúdos elaborados por terceiros, citados e referenciados nesta obra não são cobertos pela licença.

PABLO DE PAULA CORREIA

**H2GÁS: UMA APLICAÇÃO WEB PARA GESTÃO DE ENTREGAS DE GÁS E
ÁGUA**

Trabalho de Conclusão de Curso apresentado
como requisito para obtenção do título de
Tecnólogo em Tecnologia Em Sistemas Para
Internet do Curso de Tecnologia Em Sistemas
Para Internet da Universidade Tecnológica
Federal do Paraná.

Data de aprovação: 04/dezembro/2025

Prof. Roni Fabio Banaszewski

Doutorado

Universidade Tecnológica Federal do Paraná – UTFPR

Prof. Diego Marczal

Doutorado

Universidade Tecnológica Federal do Paraná – UTFPR

Prof. Emerson André Fedechen

Doutorado

Universidade Tecnológica Federal do Paraná – UTFPR

GUARAPUAVA

2025

AGRADECIMENTOS

Gostaria de agradecer, em primeiro lugar, meus pais Paulo e Ivete, que sempre estiveram ao meu lado, incentivando e apoiando nos momentos em que mais precisava, mesmo à distância. Registro minha gratidão à minha tia e à minha avó, que gentilmente me proporcionaram moradia e apoio durante toda a minha trajetória acadêmica, contribuindo para que eu alcançasse este importante marco. Estendo também meus agradecimentos a toda a minha família, pelo carinho, compreensão e incentivo ao longo dessa jornada.

Não poderia deixar de agradecer a todos os meus amigos que me acompanharam ao longo dessa caminhada, sempre prontos para ajudar, compartilhando aprendizados e oferecendo apoio em momentos difíceis.

Agradeço ao meu orientador, Prof. Dr. Roni Fabio Banaszewski, por toda sua ajuda e incentivo durante o desenvolvimento desse trabalho, sempre pronto para ajudar no que precisasse. Também agradeço a todos os meus professores do curso que compartilharam seus conhecimentos em sala de aula, contribuindo não apenas para minha formação acadêmica, mas também para meu crescimento profissional.

Por fim, expresso minha gratidão a todos que contribuíram, direta ou indiretamente, para a realização deste trabalho e para a conquista deste importante objetivo.

“É que a vida é uma estrada e eu tô pisando
fundo
De braços abertos pro mundo
Ó, estamos de frente pro mar, você teme se
molhar, eu mergulho
Viver não me assusta mais”

BK' – Só Eu Sei

RESUMO

Este trabalho apresenta o desenvolvimento do H2Gás, uma aplicação web voltada para facilitar a gestão de pedidos e entregas de gás e água, com foco em pequenas cidades, onde as opções tecnológicas para esses serviços ainda são limitadas. O projeto foi motivado pela necessidade de uma solução digital que atendesse tanto aos consumidores quanto aos pequenos distribuidores locais, oferecendo uma plataforma que possibilitasse a consulta de produtos, comparação de preços, realização de pedidos e acompanhamento do status das entregas. O H2Gás busca modernizar o processo de compra e entrega desses itens essenciais, promovendo maior eficiência para os distribuidores e conveniência para os clientes. A aplicação foi desenvolvida em formato de *Minimum Viable Product* (Produto Mínimo Viável) (MVP), contemplando funcionalidades como cadastro e controle de produtos e preços, além de um painel administrativo para o gerenciamento das distribuidoras e das cidades atendidas. Com isso, o H2Gás contribui para a digitalização dos serviços locais, fortalecendo o comércio regional e ampliando o acesso da população a soluções tecnológicas simples e acessíveis.

Palavras-chave: entrega de gás; entrega de água; tecnologia de entrega; gestão de pedidos; distribuidoras.

ABSTRACT

This work presents the development of H2Gás, a web application designed to facilitate the management of gas and water delivery orders, focusing on small cities where technological solutions for these services are still limited. The project was motivated by the need for a digital platform that could serve both consumers and small local distributors, offering features such as product browsing, price comparison, order placement, and delivery status tracking. H2Gás aims to modernize the purchasing and delivery process of these essential items, promoting greater efficiency for distributors and convenience for customers. The application was developed as a Minimum Viable Product (MVP), including functionalities such as product and price management, as well as an administrative panel for managing distributors and the cities served. In this context, H2Gás contributes to the digitalization of local services, strengthening regional commerce and expanding the population's access to simple and accessible technological solutions.

Keywords: gas delivery; water delivery; delivery technology; order management; distributors.

LISTA DE FIGURAS

Figura 1 – Exemplo de quadro Kanban no GitHub Projects.	20
Figura 2 – Ciclo do modelo GitFlow.	23
Figura 3 – Análise geral do projeto	32
Figura 4 – Landing page	34
Figura 5 – Tela de login	36
Figura 6 – Telas de registro	36
Figura 7 – Tela de ativação de usuários	37
Figura 8 – Arquitetura inicial do Banco de Dados	38
Figura 9 – Arquitetura final do Banco de Dados	40
Figura 10 – Landing page finalizada	43
Figura 11 – Estrutura API Backend	45
Figura 12 – Tela de cadastro de cidades	49
Figura 13 – Tela de gerenciamento dos fornecedores/distribuidores.	49
Figura 14 – Telas de produtos	51
Figura 15 – Telas de visualização de fornecedores e produtos	52
Figura 16 – Telas de fluxo de pedidos.	54
Figura 17 – Tela de pedidos recebidos pelo fornecedor/distribuidor	55
Figura 18 – Tela de pedidos realizados pelo cliente	58

LISTA DE TABELAS

Tabela 1 – Histórias de Usuário do Sistema	33
Tabela 2 – Funcionalidades previstas para versões futuras do sistema	66

LISTAGEM DE CÓDIGOS FONTE

Listagem 1 – Teste de unidade: busca de usuário por ID válido.	47
Listagem 2 – Teste de unidade: exceção ao buscar usuário com ID inexistente. . .	47
Listagem 3 – Método de cálculo do valor total do pedido	53
Listagem 4 – Enum de status de pedido.	56
Listagem 5 – Exemplo simplificado de workflow para deploy automatizado na AWS.	60

LISTA DE ABREVIATURAS E SIGLAS

Siglas

ACID	Atomicidade, Consistência, Isolamento e Durabilidade
AWS	<i>Amazon Web Services</i> (Serviços Web da Amazon)
CD	<i>Continuous Deployment</i> (Entrega Contínua)
CI	<i>Continuous Integration</i> (Integração Contínua)
CRUD	<i>Create, Read, Update, Delete</i> (Criar, Ler, Atualizar e Excluir)
DDL	<i>Data Definition Language</i> (Linguagem de Definição de Dados)
DevOps	<i>Development and Operations</i> (Desenvolvimento e Operações)
DML	<i>Data Manipulation Language</i> (Linguagem de Manipulação de Dados)
DQL	<i>Data Query Language</i> (Linguagem de Consulta de Dados)
DTO	<i>Data Transfer Object</i> (Objeto de Transferência de Dados)
GLP	Gás Liquefeito de Petróleo
HTTP	<i>HyperText Transfer Protocol</i> (Protocolo de Transferência de Hipertexto)
JPA	<i>Java Persistence API</i> (API de Persistência em Java)
JWT	<i>JSON Web Token</i> (Token Web JSON)
MoSCoW	<i>Must Have, Should Have, Could Have, Won't Have</i> (Técnica de Priorização de Requisitos)
MVC	<i>Model–View–Controller</i> (Modelo–Visão–Controlador)
MVP	<i>Minimum Viable Product</i> (Produto Mínimo Viável)
SQL	<i>Structured Query Language</i> (Linguagem de Consulta Estruturada)
TDD	<i>Test-Driven Development</i> (Desenvolvimento Orientado a Testes)

SUMÁRIO

1	INTRODUÇÃO	12
1.1	Justificativa	14
1.2	Objetivos	15
1.2.1	Objetivo geral	15
1.2.2	Objetivos específicos	15
2	TRABALHOS RELACIONADOS	16
2.1	Aplicativos de Delivery de Alimentos e Serviços Genéricos	16
2.2	Aplicativos de Delivery específico de gás	17
2.3	Considerações	18
3	REFERENCIAL TEÓRICO	19
3.1	Processo de Desenvolvimento	19
3.1.1	Scrum	19
3.1.2	Kanban	20
3.1.3	Desenvolvimento Orientado a Testes	21
3.1.4	Monorepo	21
3.2	Ferramentas de desenvolvimento	22
3.2.1	Git	22
3.2.2	GitFlow	22
3.2.3	GitHub	23
3.2.4	GitHub Projects	24
3.3	Tecnologias do lado do cliente	24
3.3.1	Angular	24
3.3.2	DaisyUI	25
3.4	Tecnologias do lado do servidor	25
3.4.1	Spring	25
3.4.2	Docker	26
3.4.3	MySQL	26
3.4.4	Amazon Web Services (AWS)	27
4	MATERIAIS E MÉTODOS	28
4.1	Materiais	28

4.2	Métodos	29
5	ANÁLISE E PROJETO DO SISTEMA	31
5.1	Análise do sistema	31
5.2	Prototipação das Telas	34
5.3	Modelagem do Banco de Dados	37
6	DESENVOLVIMENTO DO SISTEMA	42
6.1	Sprint 0	42
6.2	Sprint 1	42
6.3	Sprint 2	46
6.4	Sprint 3	48
6.5	Sprint 4	50
6.6	Sprint 5	50
6.7	Sprint 6	52
6.8	Sprint 7	55
6.9	Sprint 8	57
6.10	Sprint 9	61
6.11	Considerações sobre o desenvolvimento	62
6.11.1	Desafios e Adaptações	62
6.11.2	Aprendizado Técnico	62
6.11.3	Deploy	63
7	CONCLUSÃO	64
7.1	Trabalhos Futuros	64
	REFERÊNCIAS	67
	APÊNDICE A FUNCIONALIDADES CLASSIFICADAS COMO SHOULD HAVE, COULD HAVE E WON'T HAVE (MOSCOW)	70
	A.1 Funcionalidades Should Have (Deveria Ter)	70
	A.2 Funcionalidades Could Have (Poderia Ter)	71
	A.3 Funcionalidades Won't Have This Time (Não Terá Desta Vez)	71

1 INTRODUÇÃO

A história dos serviços essenciais nas residências no Brasil reflete o avanço gradual da infraestrutura e das tecnologias voltadas para o bem-estar da população. Em meados do século XX, o fornecimento de eletricidade para as casas trouxe conforto e segurança às famílias, enquanto o acesso à água encanada nas residências representou um marco significativo de saúde pública e praticidade. Bastava abrir a torneira para que as famílias tivessem acesso a água para beber, cozinhar e se higienizar. Em algumas cidades brasileiras, a água potável ainda chega às torneiras até hoje, embora, com o tempo, devido à poluição e à falta de tratamento em diversas áreas, o consumo direto da água encanada tenha se tornado um problema.

Esse avanço nos serviços de luz e água, contudo, não foi acompanhado por uma solução prática para o gás, especialmente após a popularização dos fogões a gás. Ao contrário da água e da eletricidade, que podiam ser fornecidas por meio de redes de distribuição conectadas diretamente às residências, o gás não pôde ser oferecido da mesma forma. Com o aumento do uso dos fogões a gás, tornou-se necessário um modelo de fornecimento seguro e acessível, que facilitasse o acesso das famílias a esse recurso essencial. Surgiram, então, as distribuidoras de gás, que estabeleceram pontos de venda próximos às residências para atender à demanda doméstica, especialmente com o Gás Liquefeito de Petróleo (GLP), que passou a ser vendido em botijões para uso doméstico (World Health Organization, 2024).

À medida que essas distribuidoras se consolidaram, uma nova demanda surgiu: a entrega de água potável para consumo, em garrafões de 20 litros, como alternativa à água da torneira que, em muitas localidades, já não era considerada potável. Com a oferta de água em garrafões, as distribuidoras passaram a fornecer não apenas gás, mas também um serviço essencial de entrega de água diretamente às famílias, simplificando o abastecimento e garantindo a segurança no consumo.

Com o crescimento das distribuidoras, tornou-se comum que os clientes precisassem ir pessoalmente às lojas para comprar o gás ou a água. Alternativamente, as distribuidoras passaram a atender pedidos por telefone, mas isso ainda exigia que o cliente realizasse uma ligação e aguardasse atendimento, o que se tornava inconveniente em casos de alta demanda, instabilidade nas ligações ou linhas ocupadas. Com a popularização dos aplicativos de mensagens, como o WhatsApp, muitas distribuidoras passaram a aceitar pedidos diretamente por mensagens, o que trouxe praticidade para os clientes. Mas ainda o atendimento apresenta limitações, como pouca flexibilidade para atender a vários clientes ao mesmo tempo, além de não oferecer uma comparação prática de preços e alternativas.

Com o avanço da tecnologia e o surgimento de aplicativos de delivery, plataformas originalmente focadas em alimentos começaram a oferecer a entrega de gás e água. Segundo France Júnior *Jornal da USP* (2021), com a pandemia do COVID 19, aumentou a popularidade desses aplicativos tornando quase que uma necessidade para estabelecimentos que antes atendiam apenas por telefone ou de forma presencial. Estes precisaram migrar para o sis-

tema de entregas por meio de aplicativos. Mesmo após o fim da pandemia, o hábito de realizar pedidos por esses sistemas se manteve.

No entanto, esse cenário é mais comum nos grandes centros urbanos, onde há um maior desenvolvimento tecnológico que acaba sendo mais fácil a adoção dessas tecnologias. Nas cidades de menor porte e menos desenvolvidas, a realidade é completamente diferente. Nessas cidades, ainda é muito comum o uso de ligações telefônicas ou atendimento presencial. Em décadas passadas, a realização de pedidos por ligações era um método eficaz e popular, mas atualmente gera problemas de comunicação. Por exemplo, as ligações podem não ser estáveis ou sofrer interferências. Ademais, elas são limitadas ao atendimento de apenas um cliente por linha telefônica, resultando em perda de venda para o estabelecimento, caso haja mais de um cliente ligando. Além disso, uma mudança de endereço ou de horário de funcionamento e até mesmo de valores dos produtos não são informadas aos clientes de forma direta. Muitas vezes, os clientes só ficam sabendo ao chegar no estabelecimento ou são comunicados indiretamente por terceiros, o que prejudica a experiência de compra e torna o processo mais desgastante para o consumidor.

A ausência de um atendente disponível no momento da ligação também representa um problema significativo. Para o pequeno empresário é um custo adicional, pois precisa contratar um funcionário fixo apenas para esse atendimento, aumentando suas despesas. Essa realidade então torna esse modelo de atendimento defasado e menos viável, reforçando a importância de novas soluções mais acessíveis.

Apesar dos avanços, os aplicativos de delivery apresentam desafios específicos para o setor de gás e água. No iFood, por exemplo, as taxas variam entre 12% e 23% sobre o valor dos pedidos, dependendo do plano escolhido pelo estabelecimento, além de uma taxa adicional de 3,2% para pagamentos online (iFood, 2024). Já o Rappi oferece taxas de 12% para planos básicos, mas cobra até 27% nos serviços de entrega completa (Rappi, 2024). Devido aos custos altos de contratar um sistema de entregas, acaba não valendo a pena, levando muitas vezes estabelecimentos à falência (MADUREIRA, 2020). Esses custos reduzem significativamente a margem de lucro dos pequenos distribuidores, tornando desvantajosa sua participação e limitando as opções de entrega para clientes em cidades menores ou regiões afastadas dos grandes centros urbanos.

Além disso, esses aplicativos não são projetados exclusivamente para o mercado de gás e água, prejudicando a experiência tanto para consumidores quanto para fornecedores, que não conseguem atender às demandas específicas desse setor. Ademais, a falta de alternativas de aplicativos de entregas que dependem da demanda ou então dos empreendedores locais cadastrarem suas lojas, o que dificilmente acontece devido ao custo operacional comentado anteriormente, torna essa talvez a maior barreira para atuar com esses sistemas.

Dessa forma, torna-se relevante o desenvolvimento de um sistema de entregas de gás e água — dois produtos essenciais para o bem-estar das famílias. Esse sistema seria mais acessível para empreendedores de cidades pequenas, cujo objetivo é facilitar o acesso a esse tipo

de serviço de entrega, e reunir essas empresas de fornecimento de gás e água para solucionar muitos dessas questões abordadas.

Além de facilitar o acesso a esses produtos, o sistema busca tornar o mercado mais competitivo com a aplicação e mais atraente para o consumidor, que muitas vezes acaba limitado a aplicações focadas em apenas uma empresa ou então em um nicho, muitas vezes sendo o alimentício. Esse projeto pretende oferecer opções mais atrativas para atender consumidores de cidades pequenas, onde as opções são frequentemente limitadas e o poder de escolha é reduzido.

O sistema busca incentivar o uso tanto por parte das distribuidoras quanto dos clientes por meio de funcionalidades de fidelização e de criação de promoções. Essas estratégias têm o objetivo de atender demandas locais e apoiar o crescimento de pequenos negócios, além de ampliar o acesso da população de pequenas cidades a ferramentas digitais relacionadas ao processo de compra. Ao adotar essas iniciativas, o sistema passa a atuar como um canal adicional para a realização de pedidos, contribuindo para a organização do mercado local e oferecendo uma alternativa estruturada para consumidores e distribuidores.

1.1 Justificativa

Atualmente, os aplicativos de delivery têm se tornado extremamente populares, facilitando a compra de produtos em geral, a descoberta de ofertas e a visibilidade de novos empreendimentos nas cidades. No entanto, muito desses aplicativos tem um foco mais na venda e entrega de alimentos, tendo suas funções voltadas para esse ramo da indústria, até podendo ser usado para entregar produtos que não sejam alimentos, porém sempre com uma certa limitação.

Além disso, as altas taxas também acabam tornando inviáveis para os pequenos empresários utilizarem esses sistemas mais populares, muitas vezes então recorrendo ao atendimento por telefone ou por grupos de mensagem, ou então apenas ao atendimento presencial. Isso leva à perda de vendas e reduz a visibilidade das distribuidoras, dificultando a captação de novos clientes.

Para enfrentar esses problemas, propõe-se o desenvolvimento de uma aplicação voltada à entrega de botijões de GLP e galões de água. O objetivo é oferecer ao usuário uma alternativa às ligações telefônicas e aos grupos de mensagens, permitindo realizar pedidos diretamente pelo sistema e acessar informações relevantes para a escolha da distribuidora, como preços praticados, ofertas disponíveis. Para as distribuidoras, a aplicação possibilita maior organização do processo de vendas e melhor acompanhamento das entregas futuras.

Essa aplicação tem como objetivo facilitar a comunicação entre clientes e distribuidoras, automatizando o processo de pedidos e reduzindo a dependência de vendas realizadas por meio de ligações telefônicas. Com isso, os consumidores passam a contar com uma forma

adicional para solicitar botijões de gás e galões de água, centralizando as informações e etapas do pedido em um único sistema.

1.2 Objetivos

1.2.1 Objetivo geral

Desenvolver uma aplicação web que facilite a comunicação entre distribuidoras de gás e água e seus clientes, permitindo a comparação de preços, a realização de pedidos e o acompanhamento das entregas de forma prática e eficiente.

1.2.2 Objetivos específicos

- Desenvolver a estrutura de atendimento regional que permita gerenciar as cidades atendidas e organizar o cadastro de distribuidores, assegurando a cobertura e controle da área de atuação.
- Criar uma funcionalidade para permitir o cadastro e atualização de produtos e preços pelos distribuidores, para que os usuários tenham acesso contínuo a informações de disponibilidade e valores atualizados dos produtos.
- Implementar recursos para o acompanhamento dos pedidos, possibilitando que distribuidores e entregadores gerenciem as solicitações de forma organizada e eficiente, com maior clareza no processo de entrega.
- Desenvolver um painel de consulta de produtos com informações completas e atualizadas, facilitando a comparação de preços e avaliações de distribuidores pelos clientes.
- Permitir que os clientes realizem pedidos e acompanhem o status das entregas em tempo real, assegurando uma comunicação ágil e transparente durante todo o processo.
- Integrar diferentes formas de pagamento e opções de entrega, oferecendo ao cliente flexibilidade para escolher o método e o horário mais convenientes.

2 TRABALHOS RELACIONADOS

Atualmente, existe uma ampla variedade de sistemas e aplicativos voltados à venda e entrega de produtos, contemplando diversas funcionalidades. No entanto, grande parte dessas soluções é direcionada especificamente ao setor alimentício ou ao uso em restaurantes, atuando como cardápios digitais ou plataformas de pedidos de refeições. Nas próximas seções serão apresentados alguns sistemas de delivery disponíveis no mercado que possuem características e funcionalidades semelhantes às propostas neste trabalho.

2.1 Aplicativos de Delivery de Alimentos e Serviços Genéricos

O iFood é uma das plataformas de delivery mais conhecidas e utilizadas no Brasil, sendo referência no setor por oferecer uma ampla variedade de restaurantes e estabelecimentos cadastrados. Atualmente, conta com milhões de usuários, entre consumidores e restaurantes parceiros, e apresenta uma interface simples e intuitiva. O cliente pode realizar buscas completas pelos estabelecimentos desejados, acompanhar o status das entregas em tempo real e, ao final do pedido, avaliar o serviço prestado (iFood, 2023).

Entretanto, as altas taxas cobradas e o foco nos grandes centros urbanos acabam afastando pequenos empresários do setor. Além disso, por não ser uma plataforma específica para a entrega de gás e água, seu uso para esse fim exige adaptações, uma vez que suas funcionalidades são voltadas principalmente ao segmento alimentício.

- **Pontos Positivos:**

- Possui uma grande variedade de opções de escolha para o cliente.
- Interface amigável para o usuário.
- Acompanhamento do pedido e seus status em tempo real.
- Bom suporte para eventuais problemas com as entregas.

- **Pontos Negativos:**

- Altas taxas cobradas aos estabelecimentos, impactando no lucro final.
- Não é específico para as entregas de gás e água, tornando o sistema confuso para o uso dos fornecedores.
- Atendendo apenas em cidades com um número grande de habitantes.

O Aiqfome é outro exemplo de aplicativo voltado principalmente para restaurantes, mas que também atua no mercado de entrega de gás e de outros tipos de produtos, ainda que seu foco principal continue sendo o setor alimentício. Com uma abordagem mais regional, a plataforma se destaca por atender cidades de menor porte, diferentemente de outras aplicações

concentradas em grandes centros urbanos (Aiqfome, 2024). No entanto, apresenta limitações semelhantes às de outras plataformas generalistas, pois não é específica para a venda de produtos que não sejam alimentos. Dessa forma, acaba oferecendo uma experiência menos completa para o usuário, com restrições em funcionalidades como promoções, agendamento de entregas e disponibilidade de gás apenas em poucas distribuidoras cadastradas.

2.2 Aplicativos de Delivery específico de gás

Atualmente, existem alguns aplicativos voltados exclusivamente para a venda e entrega de gás, que se diferenciam das plataformas generalistas por oferecerem funcionalidades específicas para esse nicho de mercado. Esses sistemas buscam modernizar o processo de compra, apresentando-se como alternativas aos tradicionais pedidos feitos por telefone.

Entretanto, grande parte dessas soluções é restrita a uma única marca ou distribuidora, limitando o alcance geográfico e a competitividade entre empresas. Um exemplo é o aplicativo da Ultragaz, desenvolvido pela fornecedora nacional de GLP com o mesmo nome, que conecta os consumidores diretamente às revendas autorizadas mais próximas (Ultragaz, 2020). A plataforma possui uma interface intuitiva e é voltada principalmente para dispositivos móveis, mas seu atendimento é restrito às localidades onde há revendedores da marca. Dessa forma, o cliente fica limitado a uma única opção de compra, sem possibilidade de comparar preços ou escolher entre diferentes distribuidoras.

Outro exemplo é o aplicativo Gás em Casa, desenvolvido pela distribuidora local Dubena Ltda, sediada em Guarapuava. Diferentemente da Ultragaz, essa aplicação atua em âmbito regional, atendendo apenas à área de cobertura da própria distribuidora. Embora ofereça praticidade para os consumidores locais, também apresenta limitações de escala e de competitividade, uma vez que não contempla outras empresas ou marcas do setor.

Pontos positivos:

- Oferecem funcionalidades específicas voltadas à venda e entrega de gás (e, em alguns casos, de água), atendendo às particularidades desse tipo de produto.
- Apresentam custos operacionais e taxas geralmente inferiores aos de plataformas de delivery generalistas, o que pode favorecer pequenas distribuidoras.
- Proporcionam suporte técnico e operacional direcionado às distribuidoras, facilitando o gerenciamento de pedidos e entregas.

Pontos negativos:

- Normalmente estão vinculados a uma única marca ou distribuidora, restringindo a concorrência e as opções de escolha para o consumidor.

- Possuem cobertura geográfica limitada, atendendo apenas regiões onde existam revendedores autorizados daquela marca.
- Reduzem a competitividade do mercado local, pois impedem a comparação de preços e a escolha entre diferentes distribuidoras.

2.3 Considerações

Ao analisar os aplicativos de *delivery* existentes, como iFood e Aiqfome, nota-se que, embora tenham revolucionado o setor alimentício, eles enfrentam limitações quando usados para entregas de gás e água. As altas taxas e a falta de funcionalidades específicas para esses produtos tornam seu uso pouco viável para pequenos distribuidores, especialmente em cidades menores.

Já os aplicativos dedicados, como Ultragaz, atendem melhor o nicho, mas geralmente são restritos a certas marcas e locais, o que limita a escolha do consumidor e a competitividade no mercado.

Essas observações reforçam a necessidade do H2Gás, uma solução específica para o setor de gás e água que seja acessível, permitindo maior praticidade aos consumidores e competitividade aos distribuidores locais, especialmente em regiões com infraestrutura limitada.

3 REFERENCIAL TEÓRICO

Para o desenvolvimento de uma aplicação *web* voltada ao gerenciamento de entregas de gás e água, faz-se necessário realizar um levantamento teórico e técnico que permita compreender os requisitos essenciais e o foco principal da solução proposta. O objetivo é facilitar o uso tanto para os clientes quanto para as distribuidoras, concentrando em um único ambiente digital o cadastro e o controle das empresas, o que simplifica a busca por ofertas e amplia a eficiência na gestão de entregas por parte dos distribuidores.

Para viabilizar essa aplicação, é fundamental considerar as diversas ferramentas disponíveis no mercado, como linguagens de programação, bibliotecas e *frameworks*. A escolha dessas tecnologias deve ser criteriosa, pois influencia diretamente a qualidade do produto final e o prazo de implementação. Além disso, é indispensável adotar boas práticas de desenvolvimento de software, garantindo que o código seja estruturado, de fácil manutenção e preparado para evoluções futuras (UFRJ, 2025).

Neste capítulo, serão apresentadas as principais tecnologias, metodologias e ferramentas empregadas no desenvolvimento da aplicação, destacando-se suas funções e contribuições para a construção do sistema *H2Gás*.

3.1 Processo de Desenvolvimento

O processo de desenvolvimento abrange todas as etapas de criação do sistema, desde o planejamento inicial até a entrega final do produto. É fundamental a adoção de métodos e técnicas que auxiliem o desenvolvedor na organização das atividades, garantindo que as funcionalidades sejam implementadas dentro dos prazos estabelecidos e com a qualidade esperada. Para isso, serão utilizadas metodologias e ferramentas de gerenciamento ágil, que visam otimizar o fluxo de trabalho e manter o controle do projeto de forma estruturada.

3.1.1 Scrum

O Scrum é um *framework* amplamente utilizado no contexto do desenvolvimento de software devido a sua flexibilidade e a capacidade de organizar desde projetos simples até os mais complexos. Seu foco está na entrega contínua de valor e na colaboração entre os membros da equipe, promovendo comunicação eficiente e transparência em todas as etapas do processo.

O método divide o desenvolvimento em ciclos chamados *sprints*, com duração previamente definida, nos quais a equipe trabalha para atingir metas específicas. Também são definidos papéis fundamentais dentro do Scrum: o *Product Owner*, o *Scrum Master* e os *Developers*. O *Product Owner* atua como representante do cliente, sendo responsável por priorizar requisitos, definir as tarefas e validar as entregas ao final de cada *sprint*. O *Scrum Master* assegura

que o método seja corretamente aplicado, apoiando a equipe no entendimento e na execução do *framework*. Já os *Developers* são responsáveis pela implementação técnica do sistema, seguindo as orientações definidas nas reuniões e pelo *Scrum Master* (CNN, 2023).

O *Scrum* também é composto por eventos que reforçam a transparência e promovem a melhoria contínua. A *Sprint Planning* marca o início de cada ciclo, quando são definidos o escopo e as metas da entrega. Durante o processo, ocorrem as *Daily Scrums*, reuniões diárias de curta duração destinadas a alinhar o progresso e resolver possíveis impedimentos. Ao final da *sprint*, realiza-se a *Sprint Review*, na qual as entregas são apresentadas, seguida pela *Sprint Retrospective*, momento em que a equipe reflete sobre o processo e identifica oportunidades de aprimoramento. Essa abordagem favorece a adaptação constante e o profissionalismo durante todas as etapas do projeto (DRUMOND, 2025).

3.1.2 Kanban

Implementado em conjunto com o *Scrum*, o *Kanban* é um método visual de gerenciamento de tarefas que tem como objetivo otimizar o fluxo de trabalho e aumentar a produtividade. Amplamente utilizado no desenvolvimento de software, também pode ser aplicado a atividades cotidianas, devido à sua simplicidade e capacidade de adaptação.

O quadro *Kanban* é estruturado em colunas que representam os estágios das tarefas, geralmente divididas em “A Fazer” (*To Do*), “Em Progresso” (*In Progress*) e “Concluído” (*Done*). No entanto, essa estrutura pode ser personalizada de acordo com as necessidades de cada equipe ou projeto, permitindo a criação de novas etapas, como “Em Revisão” (*In Review*), conforme ilustrado na Figura 1 (TOTVS, 2023).

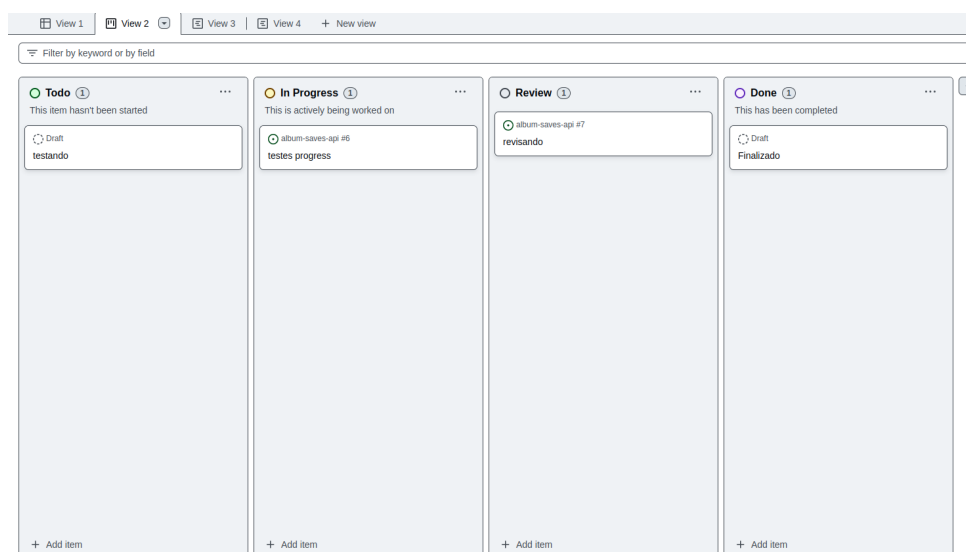


Figura 1 – Exemplo de quadro Kanban no GitHub Projects.

Fonte: Autoria própria (2025).

Por meio dessa abordagem, toda a equipe consegue visualizar o andamento das tarefas em tempo real, identificando com facilidade quais atividades estão pendentes, em desenvolvimento ou concluídas. Além disso, o método permite detectar gargalos e propor soluções mais ágeis, contribuindo para um gerenciamento mais eficiente do projeto.

3.1.3 Desenvolvimento Orientado a Testes

O *Test-Driven Development* (Desenvolvimento Orientado a Testes) (TDD) é uma metodologia de desenvolvimento de software, que visa priorizar a escrita de testes antes da implementação do código-fonte. Esse processo segue um ciclo conhecido como *Red-Green-Refactor*, onde primeiro se escreve um pequeno teste que irá falhar (*Red*), depois é implementado um código para ser aprovado no teste anterior (*Green*), e por fim é refatorado o código para melhorar sua estrutura, deixando-o mais legível e rápido (*Refactor*) (STEINFELD, 2024).

Essa abordagem reduz significativamente as chances de erros, pois garante que cada funcionalidade seja testada desde o início, permitindo identificar problemas antes mesmo da implementação completa. Além disso, o TDD facilita a manutenção e aumenta a confiabilidade ao desenvolver novas funcionalidades.

O uso de ferramentas como *JUnit* e *Mockito*, amplamente empregadas na escrita de testes automatizados, torna possível validar cada componente do sistema de forma contínua. Isso assegura que o comportamento esperado seja preservado ao longo das iterações do desenvolvimento. Embora o TDD demande um tempo inicial maior, sua aplicação resulta em um código mais seguro, modular e sustentável, favorecendo a evolução futura da aplicação (GILL, 2024).

3.1.4 Monorepo

A abordagem Monorepo consiste em armazenar o código-fonte de múltiplos serviços ou aplicações em apenas um repositório. O Monorepo permite que todas as partes do projeto compartilhem um mesmo espaço de versionamento e também suas configurações. Grandes empresas como Google, Facebook e Microsoft utilizam essa estratégia para facilitar a colaboração entre suas equipes (Eagle, Alex et al, 2025).

O uso do Monorepo oferece muitos benefícios para equipes de desenvolvimento principalmente em projetos com múltiplos serviços. Ao centralizar todas as partes do sistema em um único repositório, é possível reduzir a complexidade de gerenciar versões entre esses serviços facilitando a integração contínua. Além disso, com um único repositório, a colaboração entre equipes se torna mais ágil e consistente nas práticas de desenvolvimento, permitindo detectar e corrigir erros de forma mais rápida.

3.2 Ferramentas de desenvolvimento

As ferramentas de desenvolvimento compreendem recursos essenciais que auxiliam na criação de sistemas de forma organizada e eficiente. Seu uso adequado contribui para a melhoria da qualidade do código, a redução de tarefas manuais e repetitivas e o aumento da produtividade da equipe. Para que esse objetivo possa ser atingido, serão apresentadas ferramentas que irão facilitar o desenvolvimento deste projeto.

3.2.1 Git

O *Git* é um sistema de controle de versão distribuído criado por Linus Torvalds em 2005, amplamente utilizado no desenvolvimento de software. Ele registra todas as modificações realizadas no código-fonte, permitindo acompanhar, comparar e reverter alterações quando necessário.

O versionamento contínuo proporcionado pelo *Git* garante a integridade do histórico do projeto e facilita a evolução do código. Mesmo em projetos com apenas um desenvolvedor, o uso do *Git* oferece benefícios como segurança, rastreabilidade e a possibilidade de testar novas implementações sem comprometer a versão principal.

Entre suas principais funcionalidades estão o rastreamento completo das mudanças, a criação de ramificações (*branches*) para desenvolvimento paralelo e a integração de versões por meio de *merges*.

3.2.2 GitFlow

O *GitFlow* é um modelo de ramificação criado para organizar o uso do *Git* durante o desenvolvimento de software, definindo um fluxo padronizado para controle de versões e integração contínua. Esse modelo estabelece um conjunto de *branches* com funções bem definidas, o que facilita a coordenação de tarefas entre vários desenvolvedores e reduz conflitos de código durante o ciclo de desenvolvimento (Atlassian, 2025).

As duas principais ramificações do *GitFlow* são a *main*, que contém o código estável e em produção, e a *develop*, que reúne as versões em desenvolvimento e teste. A partir delas, são criadas ramificações auxiliares, como as de *feature* (para o desenvolvimento de novas funcionalidades), *release* (para preparação de versões que serão lançadas) e *hotfix* (para correções emergenciais diretamente na produção). A Figura 2 ilustra o fluxo de trabalho definido por esse modelo.

Nesse fluxo, a *branch main* contém as versões já publicadas, enquanto a *develop* concentra o código em fase de testes e integração. A partir dela, criam-se ramificações *feature* para o desenvolvimento de funcionalidades específicas. Quando o conjunto de funcionalidades está

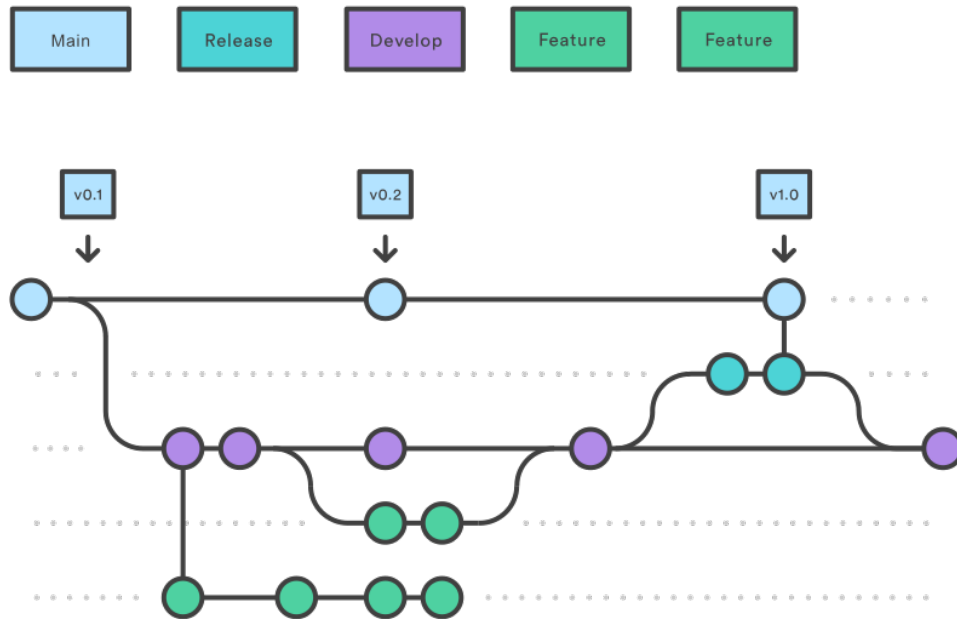


Figura 2 – Ciclo do modelo GitFlow.

Fonte: Adaptado de Atlassian (2025).

pronto, gera-se uma *release* a partir da *develop*, preparando a versão para o lançamento. Após a validação, realiza-se o *merge* da *release* na *main*, acompanhada de uma marcação de versão (*tag*), garantindo um fluxo contínuo e bem estruturado entre as diferentes etapas do projeto.

Mesmo em projetos com apenas um desenvolvedor, o uso do *GitFlow* auxilia na manutenção da organização e do histórico das alterações, permitindo separar fases de teste e produção de forma clara. Essa estrutura favorece a rastreabilidade e o controle de versões, facilitando futuras correções ou aprimoramentos no sistema (Atlassian, 2025).

3.2.3 GitHub

O *GitHub* é uma plataforma de desenvolvimento colaborativo baseada no sistema de controle de versão *Git*, que permite hospedar repositórios em nuvem e gerenciar projetos de software de maneira integrada. Por meio dela, é possível compartilhar, armazenar e acompanhar as modificações realizadas no código-fonte, criando um fluxo contínuo de colaboração entre desenvolvedores. Esse modelo de trabalho é um dos principais diferenciais do *GitHub*, pois viabiliza o desenvolvimento coletivo, seja entre membros de uma mesma equipe, seja entre colaboradores de diferentes partes do mundo interessados em contribuir com um projeto (GitHub, 2025).

Entre as principais ferramentas oferecidas pela plataforma estão o *GitHub Actions*, para automação de processos; o *GitHub Projects*, para o gerenciamento visual de tarefas; e os *Pull Requests*, que permitem a revisão de código e o controle de integração de alterações. Esses recursos tornam o fluxo de trabalho mais estável e organizado, reduzindo falhas e promovendo a colaboração em tempo real entre os participantes do projeto.

3.2.4 GitHub Projects

O *GitHub Projects* é uma ferramenta integrada ao GitHub, que tem como objetivo tornar mais simples e organizado o trabalho do desenvolvedor de forma visual. Ele oferece funcionalidades como quadros Kanban para gerenciamento de tarefas, tabelas personalizáveis e uma visualização em formato de *RoadMap*. Essa flexibilidade possibilita que equipes colaborem conforme as demandas do projeto, promovendo maior alinhamento e produtividade (Github, 2025).

Uma das grandes vantagens do *GitHub Projects* é a possibilidade de automatizar tarefas repetitivas utilizando o *GitHub Actions*, ou então integrando diretamente com *Issues* e *pull requests*. Isso reduz a necessidade de ferramentas externas de gerenciamento de projetos. Além disso, também há funções para a criação de gráficos que oferecem uma visão do progresso do projeto e ajudam em novas decisões estratégicas que precisam ser tomadas no decorrer do desenvolvimento, tornando o *GitHub Projects* uma ferramenta versátil para projetos de qualquer escala (Github, 2025).

3.3 Tecnologias do lado do cliente

O lado do cliente, o *frontend*, é a camada que interage diretamente com o usuário, sendo responsável pela criação da interface visual e pela experiência de uso. É nessa camada que os elementos visuais como botões, menus, formulários de cadastro e layouts serão construídos. Para o desenvolvimento do *frontend*, utilizam-se tecnologias específicas, como bibliotecas ou *frameworks*, que facilitam a criação de interfaces modernas e responsivas, que serão então apresentadas nas próximas seções.

3.3.1 Angular

O *Angular* é um *framework* de desenvolvimento *open-source* criado e mantido pela Google, amplamente utilizado na construção de aplicações web dinâmicas. Baseado em *TypeScript*, ele adota uma arquitetura modular, fundamentada no uso de componentes, o que favorece a organização e a reutilização do código.

Entre suas principais funcionalidades estão o sistema de roteamento, que possibilita uma navegação fluida entre as páginas, a injeção de dependências, que facilita a manutenção e escalabilidade do projeto, e o suporte integrado à comunicação com *Application Programming Interfaces* (Interfaces de Programação de Aplicações) por meio de serviços *HyperText Transfer Protocol* (Protocolo de Transferência de Hipertexto) (HTTP). Devido à sua robustez, documentação abrangente e comunidade ativa, o *Angular* consolidou-se como uma das principais ferramentas para o desenvolvimento de interfaces modernas e responsivas (Angular Developers, 2025).

3.3.2 DaisyUI

O *DaisyUI* é um *framework* CSS construído sobre o *Tailwind CSS*, projetado para simplificar o desenvolvimento de interfaces com uma abordagem prática e eficiente. Ele expande as funcionalidades do *Tailwind* ao fornecer componentes pré-construídos, porém altamente personalizáveis, que aceleram o processo de criação de layouts modernos e responsivos.

Entre suas principais vantagens estão a alta compatibilidade com o ecossistema *Tailwind*, a flexibilidade de personalização e a ampla biblioteca de componentes prontos, como botões, cartões, modais e tabelas. Esses recursos tornam o código mais limpo, organizado e de fácil manutenção, reduzindo o tempo necessário para o desenvolvimento de interfaces visuais (Saadeghi, Pouya, 2025).

3.4 Tecnologias do lado do servidor

O *backend*, que também é conhecido como o lado do servidor, atua como uma espécie de cérebro da aplicação. Ele é responsável por receber todas as solicitações enviadas pelo *frontend* por meio de requisições, gerencia todo o armazenamento da aplicação, onde é possível realizar manipulações dos dados, definir rotas que o sistema utilizará e também organizar todo o processo de segurança dessa aplicação.

3.4.1 Spring

O *Spring* é um dos ecossistemas mais completos e utilizados para o desenvolvimento de aplicações em Java. Ele fornece uma série de módulos que abrangem diferentes aspectos do desenvolvimento, como segurança, persistência de dados, injeção de dependências e criação de APIs.

Entre suas principais extensões está o *Spring Boot*, que simplifica a configuração e a execução de aplicações ao fornecer autoconfiguração, dependências pré-definidas e servidores embutidos, como Tomcat ou Jetty. Esse recurso reduz o tempo de implantação e facilita a criação de aplicações independentes e de fácil manutenção (IBM, 2025).

Para segurança e autenticação, destaca-se o módulo *Spring Security*, que pode ser integrado ao uso de *JSON Web Token* (Token Web JSON) (JWT) (*JSON Web Token*), permitindo autenticação baseada em tokens. Essa abordagem garante uma comunicação segura entre cliente e servidor, eliminando a necessidade de armazenamento de sessões no servidor.

Outro módulo amplamente utilizado é o *Spring MVC*, que adota o padrão arquitetural *Model–View–Controller* (Modelo–Visão–Controlador) (MVC), separando a aplicação em três camadas: *Model* (lógica de negócios e dados), *View* (interface do usuário) e *Controller* (controle

das requisições). Essa divisão facilita a manutenção, promove a escalabilidade e permite a criação de aplicações organizadas e de fácil evolução (Spring, 2024).

O ecossistema *Spring* também oferece integração nativa com ferramentas de persistência, como *Java Persistence API* (API de Persistência em Java) (JPA) e *Hibernate*, para mapeamento objeto-relacional, além de suporte a mensageria e APIs REST. Devido à sua flexibilidade, documentação extensa e ampla comunidade, o *Spring* é amplamente utilizado no desenvolvimento de aplicações corporativas e sistemas web modernos.

3.4.2 Docker

O Docker é uma ferramenta que permite criar, distribuir, executar e testar aplicações em *containers*. O *container* Docker permite que aplicações funcionem de forma independente, podendo o desenvolvedor criar e executar as bibliotecas, ferramentas e todos os elementos necessários para que a aplicação seja executada, garantindo que o software funcione de forma idêntica em qualquer infraestrutura. O principal objetivo dos *containers* é a independência da execução da aplicação, tornando mais flexível sua implementação e a utilização de infraestruturas variadas (Redhat, 2023).

A base do Docker consiste em imagens e *containers*, que contêm tudo que a aplicação precisa para funcionar e a partir dessa imagem o Docker cria os *containers*, para o gerenciamento dessas imagens é utilizado arquivos chamados Dockerfiles, que descrevem passo a passo o ambiente e as instruções necessárias para criar um container. Além disso, o Docker desempenha um papel fundamental em pipelines de *Continuous Integration* (Integração Contínua) (CI) e *Continuous Deployment* (Entrega Contínua) (CD), práticas amplamente adotadas no desenvolvimento moderno. Durante a CI, containers são utilizados para executar testes automatizados, garantindo qualidade do código e também detectando possíveis problemas. Na etapa de CD, imagens Docker são geradas e armazenadas, podendo ser rapidamente implantadas em ambientes de produção. Algumas ferramentas como *Jenkins*, *GitLab* e *GitHub Actions*, oferecem suporte nativo ao Docker, permitindo que seja integrado em todo o fluxo de trabalho.

3.4.3 MySQL

O *MySQL* é um de código aberto amplamente utilizado por sua confiabilidade, desempenho e compatibilidade com diversas linguagens de programação. Ele armazena os dados em tabelas compostas por linhas e colunas, permitindo consultas eficientes e o estabelecimento de relacionamentos entre diferentes conjuntos de informações (Oracle, 2025).

Seu funcionamento baseia-se na linguagem *Structured Query Language* (Linguagem de Consulta Estruturada) (SQL), utilizada para manipulação e consulta de dados. Entre os principais grupos de comandos estão: *Data Definition Language* (Linguagem de Definição de Dados)

(DDL), responsável pela criação e modificação de estruturas de banco; *Data Manipulation Language* (Linguagem de Manipulação de Dados) (DML), que permite inserir, atualizar e excluir registros; e *Data Query Language* (Linguagem de Consulta de Dados) (DQL), usada para consultas, como o comando *SELECT*. Além disso, o *MySQL* oferece suporte a transações com propriedades Atomicidade, Consistência, Isolamento e Durabilidade (ACID), garantindo integridade e segurança durante operações complexas e prevenindo falhas em processos de escrita e leitura de dados.

3.4.4 Amazon Web Services (AWS)

A *Amazon Web Services* (Serviços Web da Amazon) (AWS) é uma das principais plataformas de computação em nuvem do mercado, oferecendo uma ampla gama de serviços de infraestrutura, como hospedagem de aplicações, bancos de dados, armazenamento, monitoramento e distribuição de conteúdo. Por meio de seus recursos escaláveis, a AWS permite que desenvolvedores implantem e gerenciem sistemas de forma eficiente, segura e com alta disponibilidade. A plataforma também fornece suporte a práticas de *Development and Operations* (Desenvolvimento e Operações) (DevOps), integrando-se facilmente a ferramentas de automação como Docker e GitHub Actions (AWS, 2025).

4 MATERIAIS E MÉTODOS

4.1 Materiais

Para o desenvolvimento desse projeto e sua gestão, foram utilizadas diversas ferramentas que auxiliaram nesse processo durante suas mais diversificadas tarefas.

Para o controle das versões e armazenamento do código-fonte, foi utilizado o *GitHub*. O *Git Flow* também foi adotado como metodologia permitindo gerenciar o fluxo de trabalho no Git dividindo em diferentes ramificações como *main*, *develop* e *feature branches*, garantindo um desenvolvimento estruturado e organizado.

O *GitHub Projects* foi utilizado como ferramenta de gerenciamento das tarefas e *sprints*, podendo organizar em seu modelo de *Boards* separados em colunas representando etapas do desenvolvimento do projeto, tornando mais claro e intuitivo o andamento do trabalho.

No processo de criação dos protótipos das telas, foi utilizado o *Figma*. Essa ferramenta facilitou a representação dos elementos visuais do sistema, permitindo assim uma maior compreensão e também ajustes na interface de forma rápida e simples.

Na modelagem do banco de dados, foi utilizado o *MySQL Workbench*, que possibilitou a criação das tabelas e dos relacionamentos necessários, além da visualização do diagrama da estrutura do banco de dados, proporcionando uma compreensão mais clara do modelo de dados.

Para o desenvolvimento do sistema, foram empregadas as seguintes tecnologias: no lado cliente, foi utilizado o *framework* Angular, escolhido por sua capacidade de criar componentes reutilizáveis agilizando o processo de desenvolvimento do programado e interfaces responsivas essenciais para uma experiência do usuário moderna e intuitiva. Já no lado do servidor, foi utilizado o *framework* Spring em conjunto com a linguagem Java, devido à sua robustez e escalabilidade.

O *Docker* foi utilizado para a criação e gerenciamento de contêineres durante o desenvolvimento e implantação do sistema. Isso permitiu isolar o ambiente de execução, garantindo assim uma maior compatibilidade, reduzindo eventuais problemas que possam ocorrer com as configurações das dependências.

Para a elaboração do código-fonte no ambiente de desenvolvimento, foram escolhidas duas IDEs. No *frontend* foi empregado o Visual Studio Code (VS Code), uma ferramenta leve e muito flexível graças às suas extensões, ideal para trabalhar com o *framework* Angular. Já no *backend*, foi utilizado o IntelliJ IDEA, conhecido por sua robustez e integração nativa com o *framework* Spring, o que facilita o desenvolvimento, depuração e gerenciamento do projeto.

4.2 Métodos

O desenvolvimento do projeto foi conduzido com base na metodologia *Scrum*. Embora o *Scrum* seja comumente aplicado em equipes, ele foi adaptado para este projeto, desenvolvido de forma individual. O orientador atuou nos papéis de *Product Owner* e *Scrum Master*, enquanto o aluno desempenhou a função de *Developer*.

As atividades foram organizadas em *sprints*, planejadas com duração de quinze dias. Esse intervalo foi considerado adequado para o desenvolvimento das funcionalidades e para a realização de ajustes quando necessário. No início de cada *sprint*, foi realizado um planejamento que definiu as tarefas prioritárias. Para o gerenciamento dessas tarefas, utilizou-se o *GitHub Projects*, que possibilitou o acompanhamento do progresso e a definição do grau de prioridade de cada requisito.

As tarefas foram organizadas nas seguintes colunas dentro de cada *sprint*, utilizando os recursos do *GitHub Projects*, o que facilitou a visualização do andamento do trabalho:

- **To Do** (A Fazer): tarefas que estão planejadas para a *sprint*, mas ainda não foram iniciadas.
- **In Progress** (Em Progresso): tarefas que estão em execução no momento.
- **Review** (Revisão): tarefas concluídas que aguardam revisão ou testes finais.
- **Done** (Concluídas): tarefas que foram finalizadas e estão prontas para entrega.

Quanto a construção do projeto foram utilizados contêineres Docker, que permitem isolar e facilitar a implantação dos componentes do projeto. A aplicação foi dividida em três principais contêineres:

- **Backend**: responsável pela execução da API desenvolvida com Spring Boot.
- **Frontend**: desenvolvido com Angular, ficará isolado em seu próprio contêiner.
- **Banco de Dados**: utilização do MySQL para armazenar os dados da aplicação.

Com essa abordagem baseada em contêineres e integrada a ferramentas como o *GitHub Actions*, a cada novo *commit* foi acionada uma *pipeline* responsável pela execução e validação de testes automatizados antes da integração com o ambiente de produção. Esse processo permitiu uma implantação mais segura, reduzindo falhas e garantindo versões mais estáveis antes mesmo da finalização do *build* e do *deploy*.

Quanto à organização do código-fonte, foi adotada a estratégia de *monorepo* (repositório único), na qual o *frontend* e o *backend* permaneceram armazenados no mesmo repositório. Essa abordagem facilitou a gestão, a implantação e a distribuição do projeto, permitindo uma

integração contínua mais eficiente. Além disso, simplificou a execução de testes e a manutenção dos componentes, garantindo a consistência do sistema.

Com base nos materiais e metodologias descritos neste capítulo, foi possível conduzir o desenvolvimento do sistema de forma estruturada e eficiente. A combinação das ferramentas e metodologias adotadas garantiu um processo de desenvolvimento flexível, escalável e de fácil manutenção, atendendo aos requisitos do projeto dentro do prazo estabelecido e possibilitando a realização de testes e aprimoramentos contínuos.

5 ANÁLISE E PROJETO DO SISTEMA

5.1 Análise do sistema

Para a análise dos requisitos do sistema, foi adotado o modelo de Histórias de Usuário, utilizando em conjunto com o *framework* Scrum. Este modelo descreve os requisitos do ponto de vista de cada um dos perfis de usuários, utilizando uma estrutura simples, porém padronizada: "**COMO** [tipo de usuário], eu **QUERO** [ação ou necessidade] **PARA** [benefício ou propósito]". Por meio dessa abordagem é possível organizar e priorizar os requisitos com foco nas necessidades de cada usuário.

Com base nesse modelo, foram detalhadas as histórias dos quatro principais tipos de usuários do sistema: Administrador, Fornecedor/Distribuidor, Cliente e Entregador, onde cada um desses usuários apresenta níveis de acesso e recursos próprios. Essas histórias estão apresentadas na Tabela 1, destacadas as funcionalidades para cada um dos perfis, onde estão sendo representadas por um ID no formato **HUXX**, onde **XX** representa um número sequencial.

Neste trabalho, o termo **distribuidor(a)** é o tecnicamente adequado para as empresas que comercializam GLP e água. Por coerência com as interfaces do sistema, o termo **fornecedor(a)** aparece em algumas telas. No contexto deste TCC, *distribuidor(a)* e *fornecedor(a)* são usados como equivalentes.

Conforme a estrutura apresentada na Figura 3, é possível ter uma visão geral do projeto e de seus quatro principais eixos. O modelo de histórias de usuário, comentado anteriormente, foi organizado de modo a atender aos principais requisitos do sistema sob a perspectiva de cada tipo de usuário. Para a priorização desses requisitos, foi utilizado o método *Must Have, Should Have, Could Have, Won't Have* (Técnica de Priorização de Requisitos) (MoSCoW), que os classifica em quatro categorias: *Must Have* (Deve Ter), *Should Have* (Deveria Ter), *Could Have* (Poderia Ter) e *Won't Have This Time* (Não Terá Desta Vez).

A figura também destaca o escopo do projeto, cujo foco é o desenvolvimento de um MVP, contemplando apenas as funcionalidades *Must Have* — consideradas essenciais para o funcionamento básico da aplicação e para o atendimento das principais necessidades dos usuários. As demais funcionalidades, enquadradas nas categorias *Should Have*, *Could Have* e *Won't Have This Time*, foram planejadas para versões futuras, de forma a permitir a evolução gradual do sistema (ver Apêndice A).

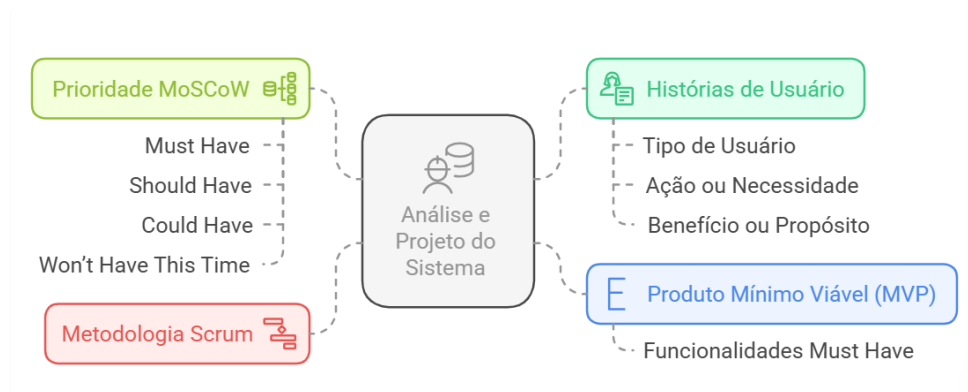


Figura 3 – Análise geral do projeto

Fonte: Autoria própria (2025).

Tabela 1 – Histórias de Usuário do Sistema

ID	Como	Quero	Para
HU01	Administrador	cadastrar as cidades em que o app estará disponível	definir a área de atuação do aplicativo
HU02	Administrador	fazer login no sistema utilizando minhas credenciais	acessar o painel de administração e gerenciar o aplicativo
HU03	Administrador	quero aprovar o cadastro de empresas distribuidoras/fornecedoras de gás e água	que os clientes possam encontrar e solicitar serviços dessas empresas
HU04	Administrador	implementar protocolos de segurança robustos	proteger as informações pessoais e financeiras dos usuários
HU05	Fornecedor	me cadastrar no aplicativo fornecendo nome, endereço e telefone	oferecer meus produtos aos clientes
HU06	Fornecedor	uma interface para cadastrar meus produtos e atualizar disponibilidade e preços	manter meu catálogo atualizado e atrair clientes
HU07	Fornecedor	fazer login no aplicativo utilizando minhas credenciais	gerenciar meus produtos, preços e pedidos dos clientes
HU08	Cliente	me cadastrar no aplicativo fornecendo nome, endereço e telefone	realizar pedidos personalizados e receber entregas no meu endereço
HU09	Cliente	fazer login no aplicativo utilizando meu e-mail e senha	acessar minha conta, realizar pedidos e acompanhar meu histórico de compras
HU10	Cliente	ver uma lista atualizada de produtos disponíveis (gás e água)	escolher o que melhor atende às minhas necessidades
HU11	Cliente	comparar preços de diferentes fornecedores	escolher a melhor opção em qualidade e preço
HU12	Cliente	ter diversas opções de pagamento (cartão, Pix, dinheiro)	escolher a mais conveniente
HU13	Cliente	receber atualizações sobre o status do meu pedido	saber quando ele será entregue
HU14	Cliente	visualizar uma lista com meus pedidos realizados	acompanhar as atualizações de status de cada um deles
HU15	Fornecedor	definir o estoque mínimo e horários de funcionamento da loja	gerenciar melhor as operações e visualizar alertas de produtos com baixo estoque

Fonte: Autoria própria (2025).

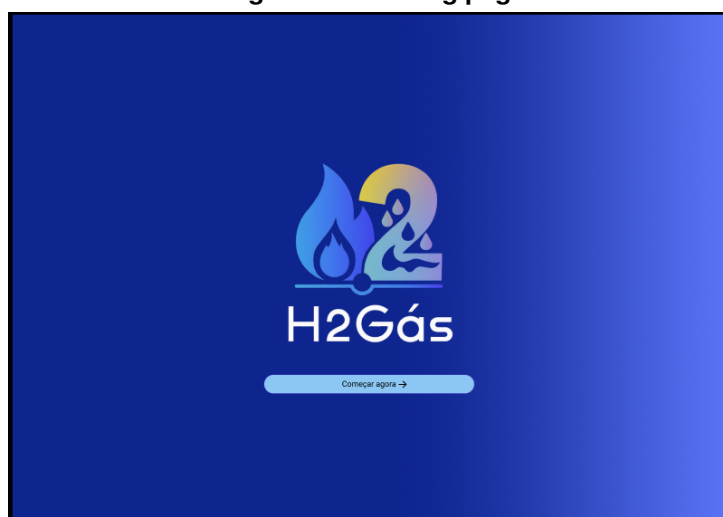
5.2 Prototipação das Telas

O sistema desenvolvido foi projetado para organizar e disponibilizar suas funcionalidades de forma clara, permitindo que o usuário navegue entre as opções de acordo com o seu perfil. O acesso é realizado por meio de uma tela de login, onde o usuário informa suas credenciais para entrar no ambiente principal. Após a autenticação, as funcionalidades apresentadas variam conforme o tipo de usuário, podendo incluir a realização de pedidos, o acompanhamento do status das entregas ou a gestão da loja. Inicialmente, os perfis são divididos em três categorias principais:

- **Administrador** : tem acesso a todas as funcionalidades do sistema, incluindo recursos exclusivos, como o gerenciamento de usuários e configurações gerais da plataforma.
- **Fornecedor**: pode gerenciar sua loja e seus produtos, tendo algumas restrições em relação a funções administrativas do sistema.
- **Cliente**: pode visualizar o catálogo de todos os distribuidores e seus produtos, além de atualizar seus dados.

A seguir, serão apresentadas as telas do sistema, começando pela *Landing Page* que pode ser vista na Figura 4, cujo objetivo é introduzir o serviço e facilitar o primeiro contato do usuário com a plataforma. Com o auxílio do Figma, apresentado na Seção 4.1, foi criado o protótipo da primeira tela do sistema, onde apresenta a logo do projeto como também seu nome, suas cores predominantes, também possibilitando ao usuário acessar a tela de *login*.

Figura 4 – Landing page



Fonte: Autoria própria (2025).

As cores escolhidas seguem uma paleta semelhante as usadas no logo da aplicação, garantindo assim uma identidade visual mais única, e também um contraste adequado com o nome do sistema. A seguir, destacam-se os principais elementos visuais apresentados na Figura 4:

- **Paleta de Cores:**

- O tom predominante é um azul gradiente, que transmite confiança, modernidade.
- O gradiente no ícone, que combina amarelo e azul, representa energia e fluidez, fazendo alusão ao principal conceito da aplicação.

- **Logotipo:**

- O ícone de chama estilizada transmite a ideia central do sistema.
- A tipografia utilizada no nome "H2Gás" é limpa e moderna, destacando-se com um tom branco que cria contraste com o fundo azul.

- **Fundo:**

- O fundo utiliza um gradiente azul, gerando um visual mais profissional e moderno. Tornando a experiência visual confortável ao usuário.

- **Botão de Ação:**

- O botão “Começar agora” utiliza um azul mais claro, com intuito de chamar atenção do usuário e destacar dos demais elementos.
- A borda arredondada e o ícone de seta tornam o botão amigável e intuitivo.

Uma vez definida a identidade visual e componentes utilizados para a *Landing Page*, as próximas telas apresentadas foram propostas para serem desenvolvidas durante a primeira Sprint. A prioridade foi implementar as funcionalidades essenciais para o funcionamento do sistema, bem como aquelas que dependem de outras para funcionar corretamente. A Tabela 1 apresenta as Histórias de Usuário relacionadas a essas funcionalidades, destacando a História de ID **HU03** e **HU05** que foram escolhidas para serem implementadas inicialmente, devido à sua importância para o funcionamento inicial da aplicação.

Após a *Landing Page* o usuário tem acesso à tela de login, podendo ser visualizada no protótipos da Figura 5. Essa tela foi projetada para ser comum a todos os tipos de perfis e consiste na única forma de acesso à plataforma. O login deve ser realizado por meio de um e-mail e senha previamente cadastrados. Caso o usuário não possua um cadastro, a tela também oferece um botão para redirecioná-lo à página de registro. Além disso, também há uma opção para direcionar o usuário à página de recuperação de senha.



Figura 5 – Tela de login

Fonte: Autoria própria (2025).

Na tela de registro, o usuário encontra um formulário onde é possível preencher com suas informações, sejam elas nome, e-mail, senha e telefone. Além disso, o usuário pode escolher o tipo de conta que deseja criar (por exemplo, "Cliente" ou "Fornecedor"). Dependendo da escolha feita, alguns campos do formulário se tornam visíveis ou ocultos, ajustando-se às necessidades específicas de cada tipo de usuário. Essa diferença de comportamento pode ser visualizada nas Figuras 6a e 6b, que ilustra as alterações no formulário de acordo com o tipo de usuário selecionado.

(a) Tela de registro de fornecedor

(b) Tela de registro padrão

Figura 6 – Telas de registro

Fonte: Autoria própria (2025).

A tela de ativação de usuários distribuidores, acessada utilizando as credências do Administrador, permite visualizar e gerenciar os cadastros pendentes ou ativos de distribuidores.

Nesta tela, é apresentada uma tabela com informações essenciais, como número de identificação, e-mail, nome, telefone, cidade, nome da empresa e o status atual do distribuidor (ativado ou pendente). Além disso, há um botão disponível para ativar o cadastro do distribuidor ou remover seu registro.

Número	Email	Nome	Telefone	Cidade	Nome empresa	Status	Action
#00001	teste@teste.com	Matt Dickerson	42 99988 7766	Guarapuava	Gás do Luigi	Ativado	 
#00002	teste@teste.com	Wiktorja	42 99988 7766	Ponta Grossa	Mario Gás	Ativado	 
#00003	teste@teste.com	Trixie Byrd	42 99988 7766	Curitiba	Yoshi Gás e Água	Pendente	 
#00004	teste@teste.com	Brad Mason	42 99988 7766	Londrina	Gás do Tio Paulo	Pendente	 

Previous 1 2 3 Next

Figura 7 – Tela de ativação de usuários

Fonte: Autoria própria (2025).

As telas apresentadas até o momento representam as funcionalidades essenciais referentes às primeiras *Sprints*. O desenvolvimento dos demais protótipos continuaram nas *Sprints* seguintes, com a implementação das funcionalidades restantes, seguindo *Backlog* do Sistema, conforme se observa na Tabela 1.

5.3 Modelagem do Banco de Dados

A modelagem inicial do banco de dados da aplicação foi elaborada com o objetivo de atender às necessidades específicas de um sistema de entrega de água e gás, considerando aspectos como escalabilidade e facilidade de manutenção. Essa primeira estrutura pode ser observada na Figura 8, que apresenta as principais tabelas e seus relacionamentos planejados durante a etapa de projeto.

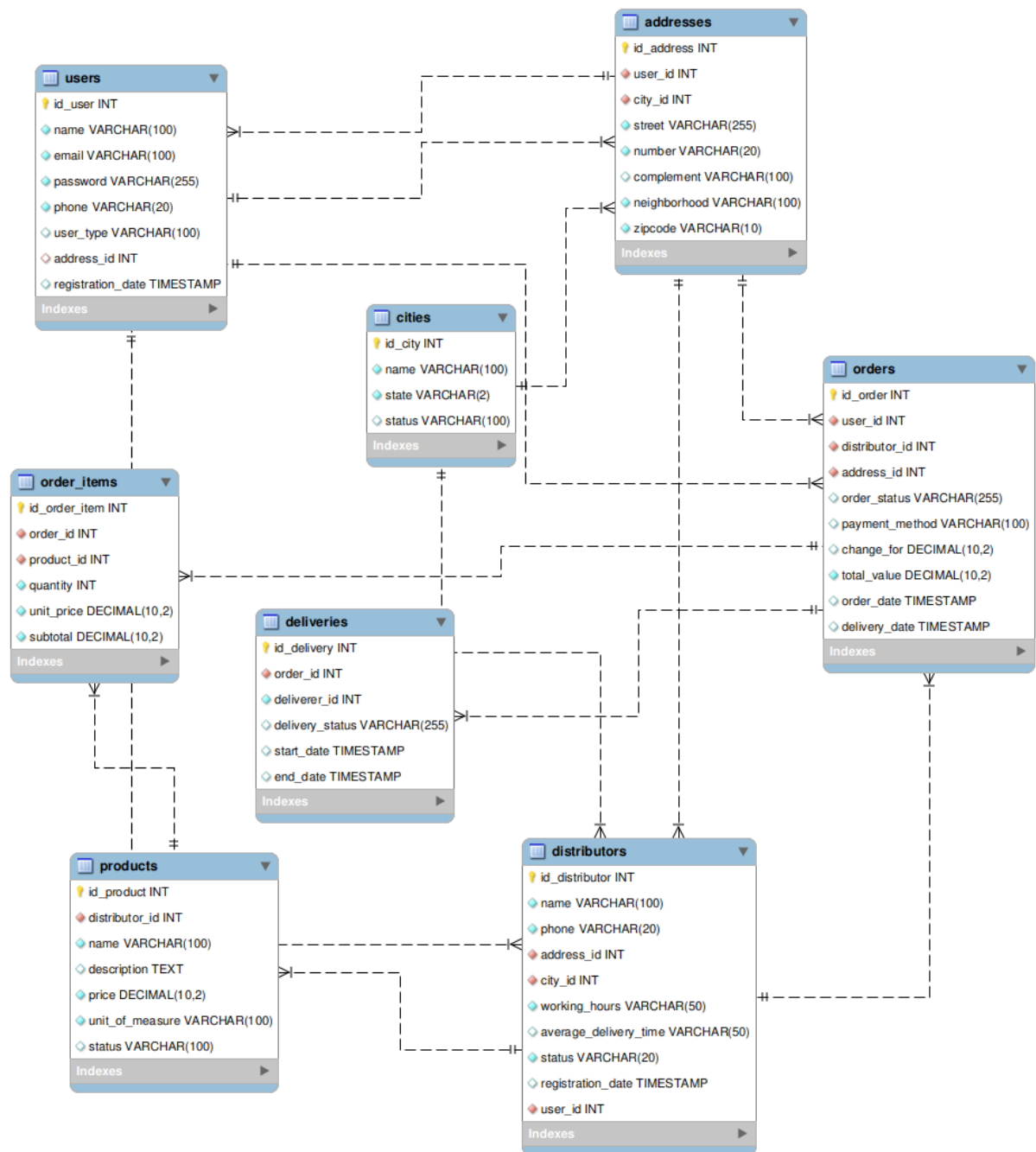


Figura 8 – Arquitetura inicial do Banco de Dados

Fonte: Autoria própria (2025).

- **users:** armazena os principais dados dos usuários do sistema, incluindo nome, e-mail, telefone, endereço, tipo de usuário (fornecedor ou cliente) e senha de acesso.
- **addresses:** armazena os endereços cadastrados pelos usuários, contendo informações como número, bairro e logradouro.
- **distributors:** registra os dados das distribuidoras cadastradas no sistema.
- **orders:** é responsável por armazenar todos os pedidos realizados pelos clientes, registrando detalhes como produto, status e data da solicitação.
- **order-items:** relaciona os produtos dentro de cada pedido, permitindo pedidos com múltiplos itens.
- **deliveries:** registra as informações da entrega de um pedido, incluindo status e data.
- **products:** lista os produtos disponíveis para venda pelas distribuidoras, como botijões de gás e garrações de água.
- **cities:** armazena as cidades atendidas pelo sistema.

Durante as etapas de desenvolvimento do projeto, a modelagem do banco de dados passou por algumas mudanças significativas para atender às novas demandas do projeto. Essa versão final pode ser visualizada na Figura 9.

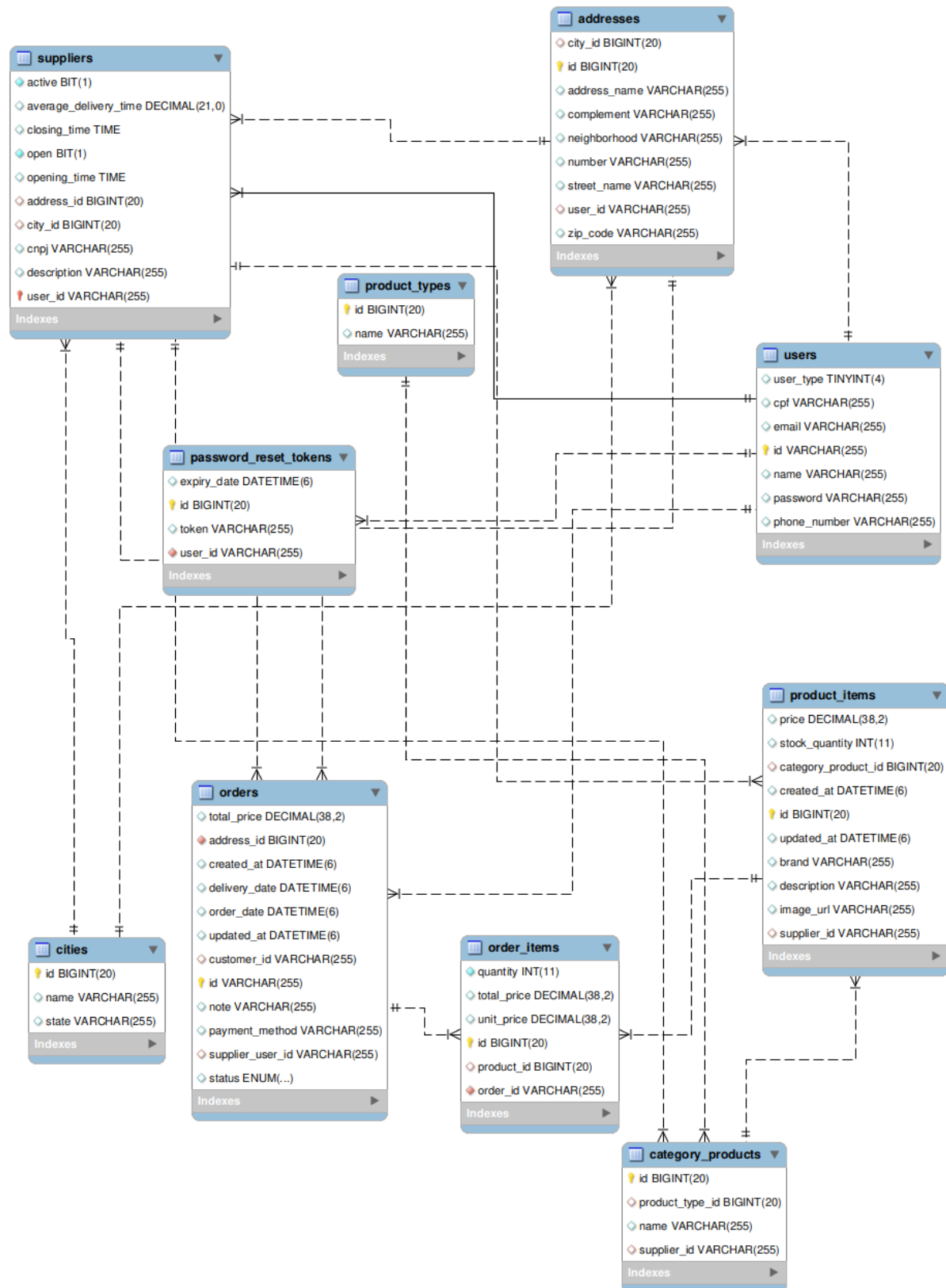


Figura 9 – Arquitetura final do Banco de Dados

Fonte: Autoria própria (2025).

Durante algumas reuniões, foram modificadas algumas tabelas:

- **Tabela *deliveries***: inicialmente projetada para registrar o histórico de entregas realizadas, foi removida após reavaliação da forma de controle das entregas nas reuniões de desenvolvimento. A funcionalidade passou para utilizar outras tabelas do projeto.
- **Tabela *distributors* → *suppliers***: a tabela de distribuidores foi mantida em termos de propósito, mas recebeu o novo nome *suppliers*, alinhando-se melhor à terminologia adotada no sistema. Além disso, optou-se por utilizar o campo *user_id* como chave de referência, unificando o gerenciamento de credenciais e informações do fornecedor com a tabela *users*.
- **Tabela *products***: a tabela original de produtos foi desmembrada em três novas entidades — *category_products*, *product_items* e *product_types*. Essa mudança foi implementada com o intuito de aumentar a escalabilidade do projeto, permitindo que, futuramente, sejam incluídos outros tipos de produtos além de água e gás, sem comprometer a estrutura do banco de dados.
- **Nova tabela *password_reset_tokens***: criada para armazenar os tokens utilizados no processo de redefinição de senha dos usuários, associando cada token a um usuário específico e registrando a data de expiração, garantindo maior segurança e controle sobre o processo de recuperação de acesso ao sistema.

6 DESENVOLVIMENTO DO SISTEMA

Neste capítulo, são detalhadas as etapas de desenvolvimento do sistema, estruturado segundo o *framework* Scrum (ver subseção 3.1.1), com adaptações ao contexto do projeto. Também são apresentadas as principais dificuldades enfrentadas ao longo de dez *sprints*, bem como a implementação das funcionalidades.

6.1 Sprint 0

Na sprint inicial, foi designado um tempo à configuração do ambiente de desenvolvimento. Nessa etapa, também foi criado o repositório do código-fonte no GitHub e também foi elaborada a criação dos arquivos *Dockerfile* e *Docker Compose* possibilitando a realização de alguns testes iniciais e preparando o projeto para um futuro processo de deploy.

Foi realizado também a configuração do *GitHub Actions*, permitindo que os testes fossem executados automaticamente sempre que fosse aberto um *Pull Request* para alguma *branch*, contribuindo então para a garantia de qualidade e a CI do código.

No *frontend*, foi realizada a configuração inicial do projeto utilizando o *framework* Angular, incluindo a instalação de dependências importantes como Tailwind CSS e DaisyUI, que foram utilizadas no decorrer de todo o desenvolvimento, a fim de criar uma padronização na interface do sistema. Durante esse processo, ocorreram alguns imprevistos devido a erros em versões de bibliotecas utilizadas, sendo necessário reinstalar e atualizar até que o ambiente se estabilizasse e ficasse adequado para o desenvolvimento.

6.2 Sprint 1

Nesta sprint foram desenvolvidas as histórias HU02¹, HU07², HU09³ e HU04⁴, referentes a implementação do login de cada um dos três tipos de usuários. No *frontend*, foi então primeiramente criado a landing page inicial como pode ser visto na Figura 10. Algumas modificações foram realizadas em relação ao design proposto anteriormente, apresentado na Figura 4, a fim de se apresentar um design mais elegante e moderno para o usuário.

¹ HU02: **Como** administrador, **quero** fazer login no sistema utilizando minhas credenciais, **para** acessar o painel de administração e gerenciar o aplicativo.

² HU07: **Como** fornecedor, **quero** fazer login no aplicativo utilizando minhas credenciais, **para** gerenciar meus produtos, preços e pedidos dos clientes.

³ HU09: **Como** cliente, **quero** fazer login no aplicativo utilizando meu e-mail e senha, **para** acessar minha conta, realizar pedidos e acompanhar meu histórico de compras.

⁴ HU04 – **Como** administrador, **quero** implementar protocolos de segurança robustos, **para** proteger as informações pessoais e financeiras dos usuários.

Figura 10 – Landing page finalizada



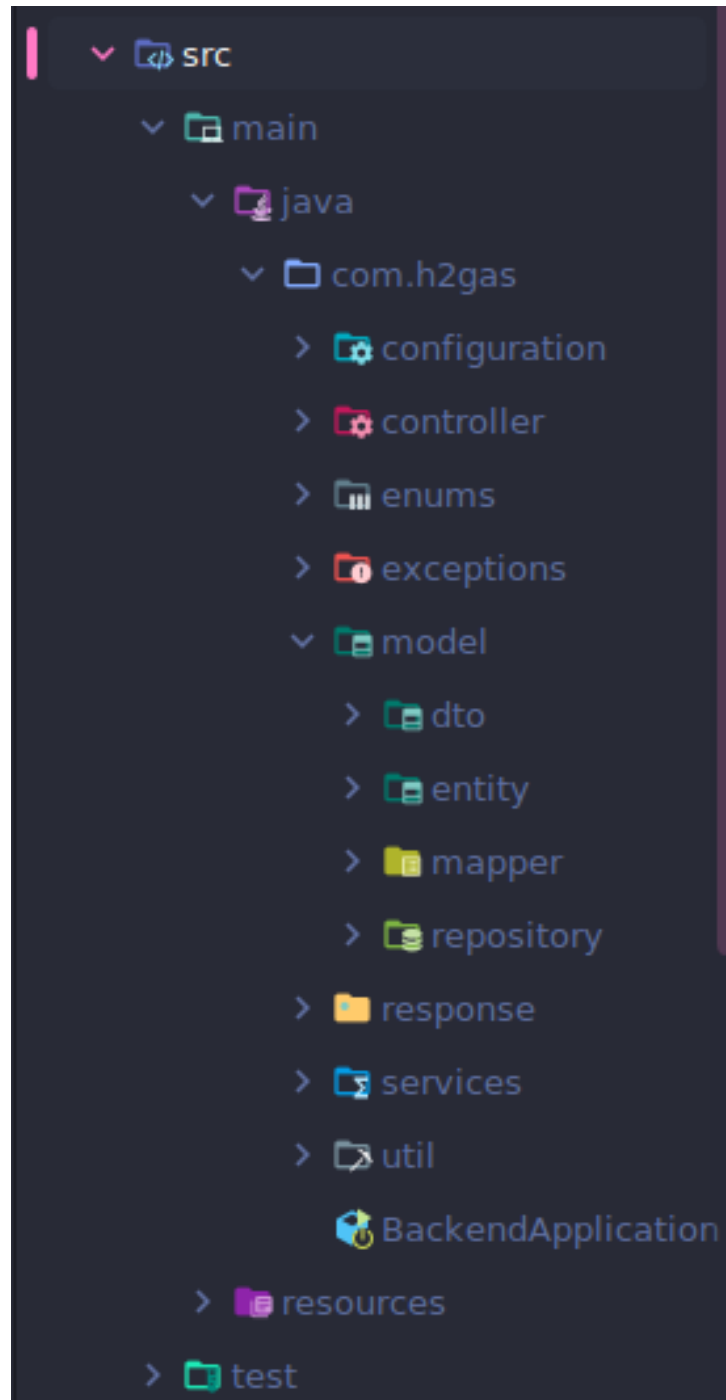
Fonte: Autoria própria (2025).

Além do desenvolvimento das telas, foram implementadas as lógicas de autenticação e cadastro, permitindo que os usuários pudessem se registrar e realizar login diretamente pelo *frontend*. No *backend*, o processo de autenticação foi desenvolvido utilizando *Spring Security* em conjunto com JWT, garantindo assim maior segurança no processo de login e possibilitando o controle de acesso às rotas da aplicação de acordo com cada um dos perfis de usuário. Essa abordagem assegura que apenas usuários autenticados e autorizados possam interagir com os recursos do sistema, e também adicionando uma camada a mais de segurança.

Durante a reunião quinzenal da *sprint*, algumas decisões envolvendo a arquitetura do projeto foram reformuladas em conjunto com o orientador. Uma das principais mudanças ocorreu no *backend*, onde a estrutura de pastas foi reorganizada. Inicialmente, não existia um diretório específico para as classes que envolviam manipulação do banco de dados, o que poderia dificultar a manutenção do código posteriormente. Com a criação da pasta `model`, foi possível adotar uma organização mais clara e alinhada às boas práticas do desenvolvimento em *Spring*, favorecendo a evolução futura do projeto. Também nesse período se optou por não seguir a ideia de separação de módulos (como usuários e cidades), uma vez que o escopo do sistema, que foi planejado como um MVP, tal abordagem traria uma camada a mais de complexidade sem realmente ser necessário para o seguimento do desenvolvimento do projeto.

A estrutura resultante para o projeto pode ser vista na figura 11, onde é possível observar os principais pacotes da aplicação que foram definidos, como `controller`, `services`, `repository` e `model` (este último subdividido em `dto`, `entity` e `mapper`), além de diretórios auxiliares como `exceptions` e `util`. Essa organização permite estabelecer uma base e também visa facilitar a separação de responsabilidades e a manutenção do código com o avanço e crescimento do sistema.

Figura 11 – Estrutura API Backend



Fonte: Autoria própria (2025).

6.3 Sprint 2

Na Sprint 2 o desenvolvimento foi focados nas histórias HU05⁵ e HU08⁶, concluindo as principais funcionalidades relacionadas ao fluxo de usuários, que engloba tanto cadastro quanto login na aplicação. Inicialmente, o projeto previa um único formulário para o registro de novos usuários, porém, durante reuniões de acompanhamento de *sprint* com o orientador, foi decidido pela separação em dois distintos formulários, um voltado ao cliente e outro ao distribuidor. Essa alteração teve como objetivo simplificar a experiência de uso e tornar o processo de cadastro mais claro para cada perfil de usuário.

No *frontend*, foram implementadas as lógicas necessárias para permitir o registro de novos usuários e login a partir das credências fornecidas. Já no *backend*, essa etapa serviu para consolidar e refinar a autenticação, com a finalização da lógica de cadastro e validação de usuários, garantindo que os dados fossem devidamente processados e validados pelo sistema.

Além do desenvolvimento das funcionalidades já comentadas, foram implementados testes específicos para essa camada de usuários. Testes unitários e de integração foram criados para validar as rotas de registro e autenticação, cobrindo o comportamento do `AuthController` e também do service de usuário, o `UserService`. Também foram feitos ajustes em testes já existentes, para aumentar a robustez e a confiabilidade do sistema. Para validar o comportamento em um ambiente mais próximo ao de produção, o sistema foi testado localmente utilizando *Docker*, permitindo simular a execução dos serviços de forma isolada e controlada.

A seguir será apresentado o teste que valida a busca de usuários pelo identificador, garantindo que um usuário existente seja corretamente retornado pelo serviço, enquanto consultas a identificadores inválidos resultam em exceção controlada. Para a implementação dos testes foram utilizadas ferramentas consolidadas no ecossistema Java, como o *JUnit 5* para a criação e execução dos casos de teste, o *Spring Boot Test* para facilitar a integração com o contexto da aplicação, e o *H2 Database* em memória para simular o repositório de dados sem necessidade de um banco externo. Essas tecnologias, em conjunto, possibilitaram a execução dos testes de forma rápida, isolada e reproduzível.

O Código 1 e 2 apresenta um teste desenvolvido para verificar o comportamento do sistema ao tentar recuperar um usuário com um identificador inexistente. Esse teste avalia se a aplicação lança a exceção esperada, garantindo que o tratamento desse cenário esteja de acordo com o comportamento definido.

⁵ HU05 – **Como** fornecedor, **quero** me cadastrar no aplicativo, fornecendo nome, endereço e telefone, **para** poder oferecer meus produtos aos clientes.

⁶ HU08 – **Como** cliente, **quero** me cadastrar no aplicativo, fornecendo nome, endereço e telefone, **para** realizar pedidos personalizados e receber entregas no meu endereço.

Listagem 1 – Teste de unidade: busca de usuário por ID válido.

```

1  @Test
2  @DisplayName("Deve retornar usuário existente ao buscar por ID
   ↪  válido")
3  public void testFindByIdUserWithExistingUser() {
4      UserDto result = userService.findByIdUser(validUserId);
5
6      assertNotNull(result);
7      assertEquals("User Teste", result.getName());
8      assertEquals("userTeste@gmail.com", result.getEmail());
9  }

```

Fonte: Autoria própria (2025).

Listagem 2 – Teste de unidade: exceção ao buscar usuário com ID inexistente.

```

1  @Test
2  @DisplayName("Deve lançar exceção ao buscar usuário com ID
   ↪  inexistente")
3  public void testFindByIdUserWithNonExistingUser() {
4      String invalidId = "94c4c7ca-6dca-4bc5-80ee-5f564918ca05";
5
6      RuntimeException exception =
7          ↪  assertThrows(RuntimeException.class, () -> {
8              userService.findByIdUser(invalidId);
9          });
10
11      assertEquals("User not found with id: " + invalidId,
12          ↪  exception.getMessage());

```

Fonte: Autoria própria (2025).

Com a conclusão desta *sprint*, a aplicação passou a dispor de um fluxo completo de autenticação e cadastro de usuários, garantindo a base necessária para que os próximos módulos pudessem ser desenvolvidos de forma integrada e segura.

6.4 Sprint 3

Na Sprint 3, o desenvolvimento focou na implementação das seguintes histórias HU01⁷, HU03⁸ e HU06⁹. Foram desenvolvidas as funcionalidades para cadastrar as cidades em que a aplicação estará disponível, aprovar o cadastro de empresas fornecedoras de gás GLP e água, e também iniciado a interface de cadastro de produtos.

Inicialmente, foi realizada uma pequena alteração no arquivo de *workflow* do *GitHub Actions*. O *.yml* anterior apresentava erros devido às novas implementações no código, e, com o aumento da quantidade de testes, foi necessário criar um cache que reduziu o tempo de execução da pipeline, evitando assim o download repetido de dependências do Maven. Essas melhorias garantiram que o *pipeline* de CI continuasse eficiente mesmo com a ampliação do conjunto de testes.

No *backend*, iniciou-se o módulo de cidades, com a criação de todas as entidades, *Data Transfer Object* (Objeto de Transferência de Dados) (DTO) e repositórios necessários para a implementação do *Create, Read, Update, Delete* (Criar, Ler, Atualizar e Excluir) (CRUD) completo das cidades atendidas. Essa etapa foi concluída com sucesso, garantindo que o administrador pudesse cadastrar e gerenciar as cidades em que a aplicação estará disponível. No *frontend*, foi iniciada a tela de cadastro de cidades atendidas, integrada à lógica implementada no *backend*. Foram criados os componentes necessários para o desenvolvimento do layout dessa interface, que permite ao administrador visualizar as cidades já cadastradas e adicionar novas, conforme ilustrado na figura 12. Além disso, foi iniciado o desenvolvimento da interface de cadastro e gerenciamento de produtos pelos fornecedores, estruturando a base para que possam atualizar preços e disponibilidade de seus itens, em conformidade com o definido na HU06.

No que se refere à HU03, foram implementadas as funcionalidades que permitem ao administrador aprovar ou rejeitar os cadastros de empresas de gás GLP e água. Essa etapa foi essencial para garantir que apenas fornecedores verificados estivessem disponíveis no sistema, aumentando assim a confiabilidade do serviço oferecido para o cliente. No *backend*, foram criados os *services* e rotas que irão verificar e atualizar o status dos cadastros, enquanto no *frontend* foi construída a interface para que o administrador pudesse gerenciar essas solicitações de forma prática e segura, como pode ser visto na Figura 13.

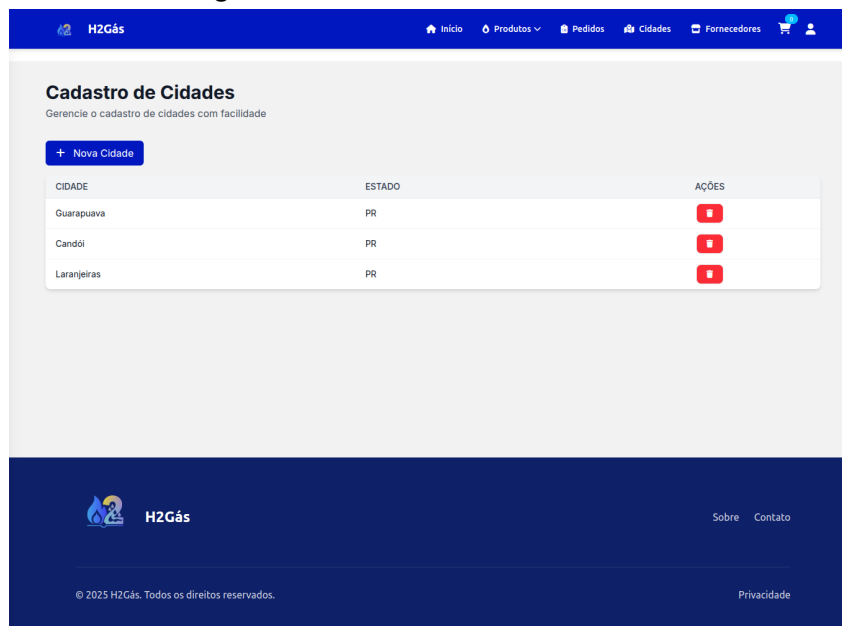
Em seguida, foi iniciado o módulo de endereços, com a criação das tabelas correspondentes e suas devidas relações com as tabelas de usuários. Durante essa implementação,

⁷ **HU01 – Cadastrar Cidades:** Como administrador, **Quero** cadastrar as cidades em que o aplicativo estará disponível e definir a área de atuação, **Para** garantir que os clientes utilizem o serviço apenas em regiões atendidas.

⁸ **HU03 – Aprovar Cadastro de Fornecedores:** Como administrador, **Quero** aprovar o cadastro de empresas distribuidoras/fornecedoras de gás GLP e água, **Para** que os clientes possam encontrar e solicitar serviços dessas empresas com segurança.

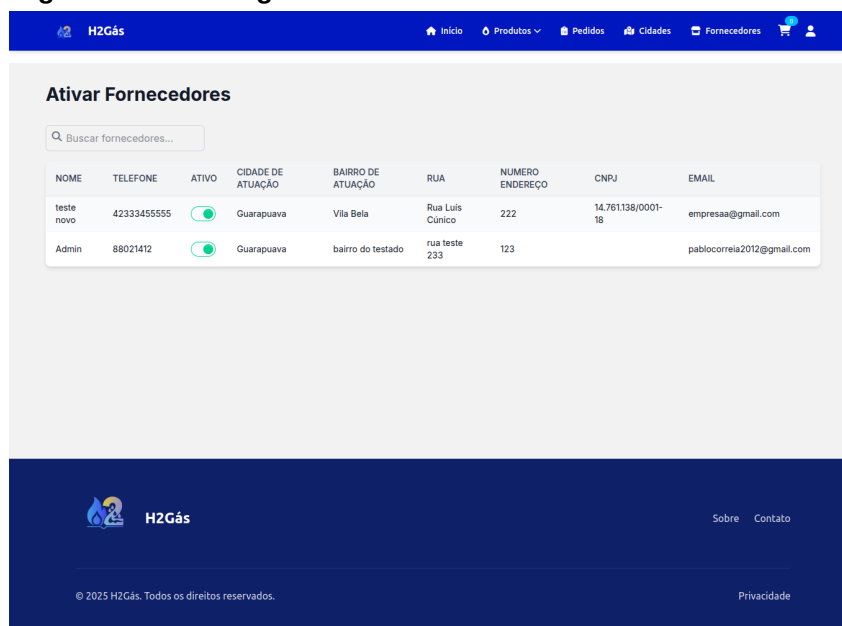
⁹ **HU06 – Gerenciar Catálogo de Produtos:** Como fornecedor, **Quero** ter uma interface para cadastrar meus produtos, atualizar disponibilidade e preços, **Para** manter meu catálogo atualizado e atrair clientes.

Figura 12 – Tela de cadastro de cidades



Fonte: Autoria própria (2025).

Figura 13 – Tela de gerenciamento dos fornecedores/distribuidores.



Fonte: Autoria própria (2025).

surgiram erros de mapeamento pelo *Hibernate*, que exigiram a recriação das tabelas para evitar falhas ao subir os relacionamentos. Após essas correções, toda a lógica do *backend* envolvendo endereços foi finalizada. Essas atividades permitiram integrar a lógica de negócios com a interface do usuário, garantindo que as funcionalidades planejadas fossem implementadas de forma consistente.

6.5 Sprint 4

Na Sprint 4, o desenvolvimento concentrou-se na continuidade da HU06¹⁰, que havia sido iniciada na *sprint* anterior. Nesta etapa, avançou-se na implementação das interfaces de gerenciamento dos produtos pelos fornecedores e também em uma estrutura mais robusta para o tratamento dos dados.

No *backend* foi realizada a modelagem completa das entidades relacionadas ao gerenciamento de produtos, com a criação das tabelas *ProductType*, *ProductItem* e *CategoryProduct*, que inicialmente foi pensada apenas como uma tabela única chamada *Products*. Porém, em reunião de revisão da *sprint*, em conjunto com o orientador, isso foi repensado para que em projetos futuros possa se expandir essa ideia e não fique restrito apenas à venda de gás GLP e água pelos usuários da aplicação.

Ainda no *backend*, foi implementada uma camada de tratamento de exceções, cujo objetivo é apresentar mensagens de erro mais claras e legíveis tanto para desenvolvedores quanto para usuários. Essa camada padroniza as respostas em caso de falhas e também evita expor informações sensíveis da aplicação, facilitando também a manutenção do projeto, além de proporcionar ao usuário mensagens de erro mais intuitivas.

Essas atividades consolidaram a HU06 e garantiram maior maturidade ao sistema, com um banco de dados mais consistente e um *backend* mais resistente a erros. Além disso, no *frontend* foi desenvolvido todo o layout das telas que permitem ao fornecedor cadastrar seus produtos, informando dados como marca, preço e disponibilidade. Essas interfaces estão ilustradas nas Figuras 14a e 14b, que representam a materialização da funcionalidade central proposta para esta *sprint*. Assim, a aplicação avança em direção a oferecer aos fornecedores um catálogo de produtos funcional e confiável, alinhado às necessidades do escopo do projeto.

6.6 Sprint 5

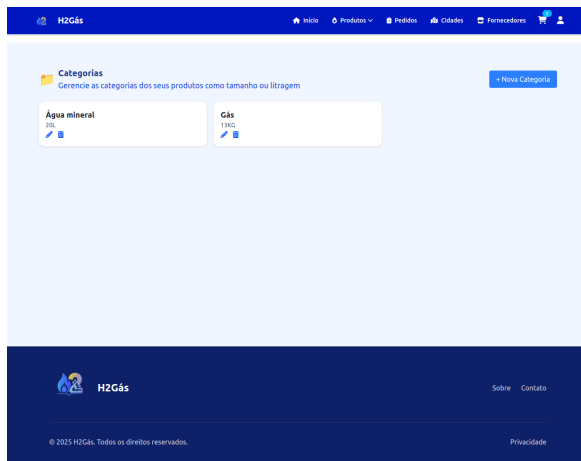
Na Sprint 5 foram implementadas novas funcionalidades voltadas ao cliente, visando aprimorar sua experiência no processo de compra. Entre elas, destacam-se as histórias de usuário que contemplam a visualização de produtos e a comparação de preços, ou seja, a HU10¹¹ e HU11¹².

Com a implementação dessas novas funcionalidades, a aplicação passou então a oferecer uma listagem dinâmica de produtos, exibindo informações como tipo, preço e disponibilidade

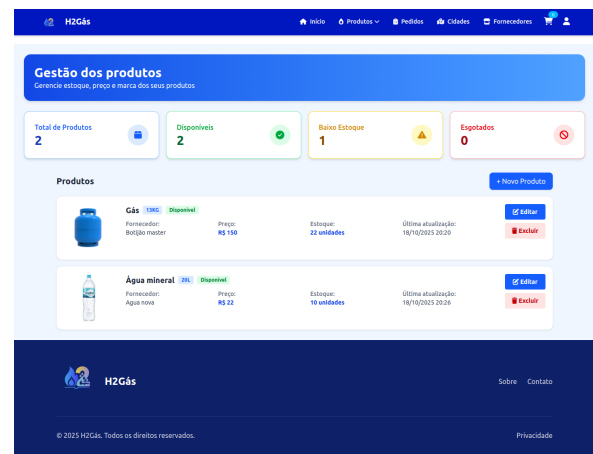
¹⁰ HU06 – **Gerenciar Catálogo de Produtos: Como** fornecedor, **Quero** ter uma interface para cadastrar meus produtos, atualizar disponibilidade e preços, **Para** manter meu catálogo atualizado e atrair clientes.

¹¹ HU10 – **Como** cliente, **quero** ver uma lista atualizada de produtos disponíveis (gás e água), **para** escolher o que melhor atende às minhas necessidades.

¹² HU11 – **Como** cliente, **quero** comparar preços de diferentes fornecedores, **para** escolher a melhor opção em qualidade e preço.



(a) Tela de gerenciamento de categorias de produtos



(b) Tela de gerenciamento geral dos produtos

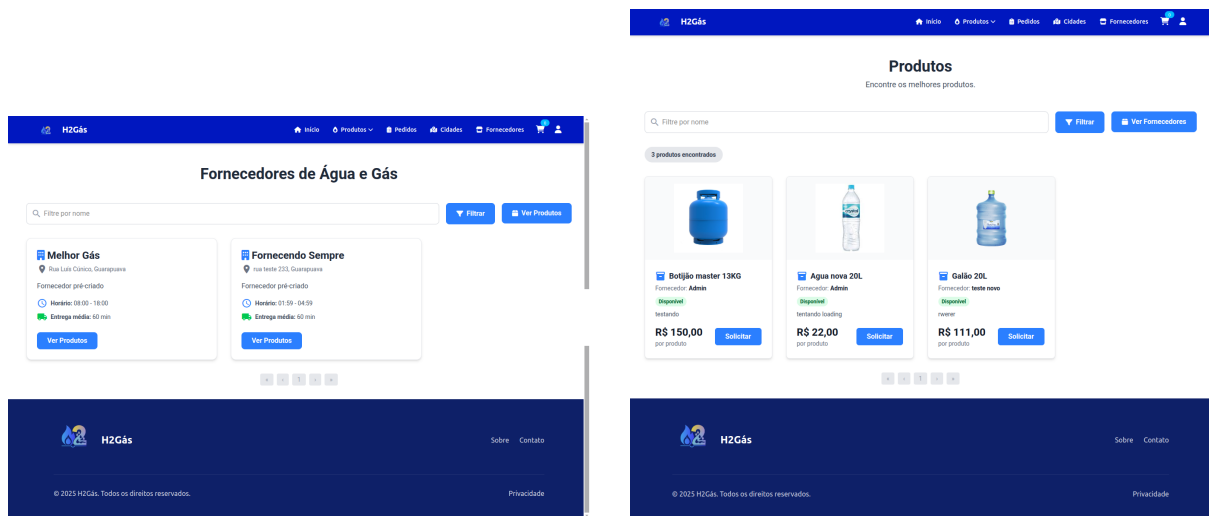
Figura 14 – Telas de produtos

Fonte: Autoria própria (2025).

e a qual fornecedora esse produto pertence. Além disso, o cliente também pode escolher entre visualizar diferentes fornecedores cadastrados, possibilitando a comparação direta de preços ou escolhendo entre o fornecedor que mais lhe agrada, o que amplia a competitividade entre os vendedores e aumentando assim o poder de escolha do consumidor.

As telas desenvolvidas nesta *sprint* reforçam a proposta de uma interface clara e objetiva, apresentando os produtos de forma organizada e facilitando a navegação. As figuras 15a e 15b ilustram as interfaces implementadas, nas quais o usuário pode alternar entre diferentes formas de visualização, optando por consultar a lista de produtos, onde é possível comparar preços e detalhes de cada item, ou a lista dos fornecedores abertos. Essa abordagem contribui para tornar a experiência do usuário mais flexível e personalizada, permitindo que o cliente acesse rapidamente as informações que considera mais relevantes durante o processo de escolha de seu produto.

Para oferecer uma listagem eficiente e escalável, foi implementado um mecanismo de paginação no *backend*, que permite controlar a quantidade de registros retornados por requisição. Essa funcionalidade foi viabilizada por meio da classe `PaginationUtil`, responsável por gerar os objetos `Pageable` conforme os parâmetros de página e tamanho informados na requisição, e pela classe `PageResponse`, criada para padronizar as respostas da API e facilitando assim o consumo dos dados pelo *frontend*. Todos os controladores responsáveis por operações de listagem passaram a adotar essa estrutura, garantindo maior consistência nas respostas e uma navegação mais fluida entre os diferentes dados apresentados na aplicação. Essa abordagem contribui muito para evitar a sobrecarga de informações e também de processamento, otimizando o tráfego entre cliente e servidor, tornando a usabilidade e a experiência geral do sistema mais leve e rápida.



(a) Tela de listagem de fornecedores/distribuidores

(b) Tela de listagem de produtos

Figura 15 – Telas de visualização de fornecedores e produtos

Fonte: Autoria própria (2025).

6.7 Sprint 6

Durante a Sprint 6, o foco inicial esteve voltado à melhoria da experiência do cliente no momento da compra, com a implementação de diferentes opções de pagamento referente a HU12¹³. Essa funcionalidade visa oferecer maior flexibilidade ao usuário, permitindo que ele escolha a forma de pagamento mais adequada às suas necessidades e ao contexto de uso do sistema.

Além dessa implementação, também foi desenvolvida a funcionalidade que permite ao fornecedor definir sua área de atuação diretamente no sistema, informando dados básicos de cadastro e o endereço da loja, facilitando o mapeamento das regiões atendidas. Outra entrega importante desta *sprint* foi a conclusão da parte de pedidos, que passou a permitir que o cliente opte entre realizar uma entrega imediata ou agendada, garantindo mais autonomia e praticidade no processo de compra.

Ainda durante o desenvolvimento desta *sprint*, também foi realizada uma reavaliação da forma de processamento dos pagamentos. Inicialmente, estava prevista a integração com o serviço do Mercado Pago, que permitiria que o cliente efetuasse seu pagamento diretamente pela plataforma. Entretanto, após discussões em algumas das reuniões com o orientador, decidiu-se que, para o escopo do projeto, que é um MVP, a abordagem mais adequada seria manter o pagamento apenas no momento da entrega. Essa mudança teve como objetivo simplificar um pouco a arquitetura inicial da aplicação, reduzindo a complexidade de integração e facilitar o processo de testes e validações. A integração com meios de pagamento digitais foi mantida como

¹³ **HU12** — Cliente ter diversas opções de pagamento (cartão, Pix, dinheiro) e escolher a mais conveniente.

uma melhoria planejada para projetos futuros, garantindo que o MVP permanecesse funcional e dentro do escopo proposto.

No que diz respeito à criação e gerenciamento dos pedidos, foi desenvolvida a lógica responsável por todo o fluxo de geração de um pedido no sistema. O serviço *OrderService* é responsável por receber as informações do pedido, mapear os dados de entrada e realizar o processamento completo antes de persistir no banco de dados. A partir disso, o sistema define também o fornecedor e o cliente relacionados a esse pedido, registra a data da solicitação, define o endereço de entrega e marca o status inicial como *PENDING*. O cálculo do valor total do pedido é realizado inteiramente no *backend*, por meio de métodos como `calculateTotalPrice()`, que soma os valores individuais de cada item (*OrderItem*). Essa decisão técnica foi tomada para reforçar a segurança da aplicação, uma vez que impede a manipulação de preços pelo lado do cliente, deixando o *frontend* responsável apenas pela exibição dos preços para o cliente. O serviço complementar *OrderItemService* também realiza o cálculo do preço total de cada item com base na quantidade e no valor unitário do produto, garantindo consistência entre os registros e o pedido final.

Listagem 3 – Método de cálculo do valor total do pedido

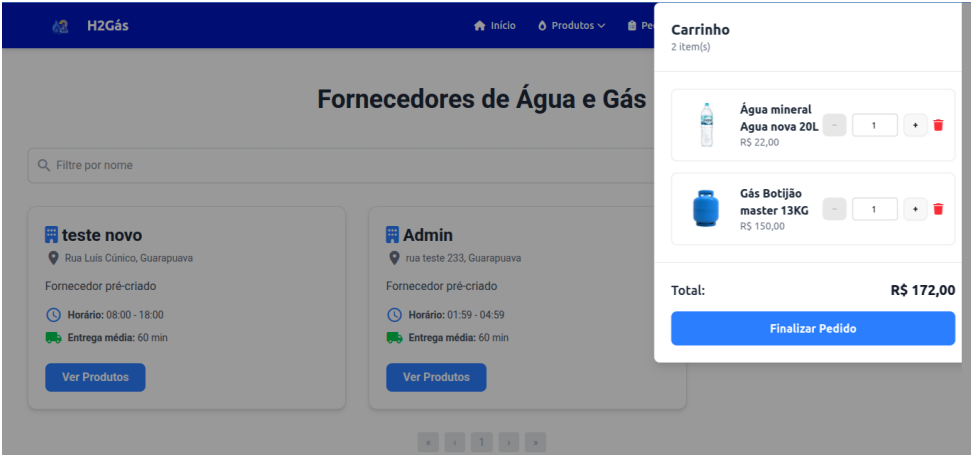
```

1 private BigDecimal calculateTotalPrice(List<OrderItem> orderItems) {
2     return orderItems.stream()
3         .map(OrderItem::getTotalPrice)
4         .reduce(BigDecimal.ZERO, BigDecimal::add);
5 }

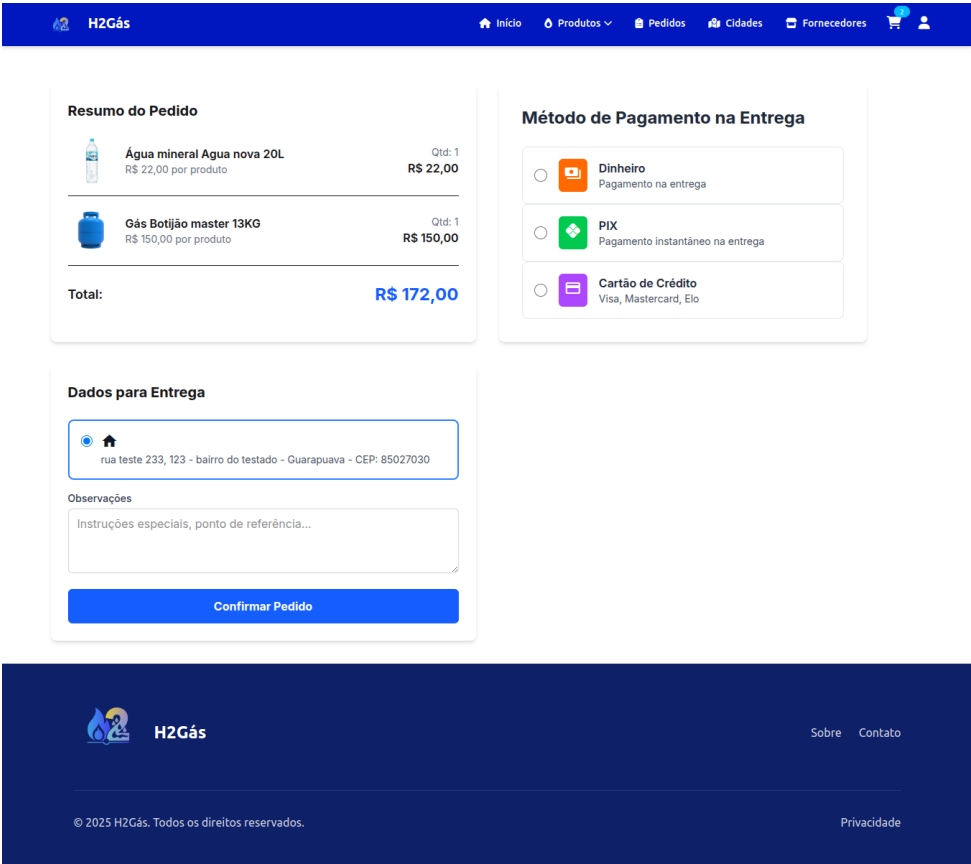
```

A listagem 3 apresenta o método responsável pelo cálculo do valor total de um pedido. Esse processamento é realizado exclusivamente no *backend*, garantindo que o cálculo não possa ser alterado manualmente pelo cliente no navegador. Dessa forma, o *frontend* atua apenas na exibição das informações, enquanto as regras de negócio e operações financeiras permanecem sob responsabilidade do servidor.

Para ilustrar as entregas desta *sprint*, a Figura 16a e 16b apresenta algumas das principais telas desenvolvidas, evidenciando a interface voltada para o cliente e para o fornecedor. As imagens destacam a simplicidade na escolha do método de pagamento, o fluxo de criação e gerenciamento de pedidos e também a criação de um carrinho dos produtos que o cliente selecionou. Essas telas permitem visualizar de forma clara como as funcionalidades implementadas foram traduzidas em elementos de interface amigáveis e funcionais.



(a) Carrinho de compras



(b) Tela de finalização de pedidos

Figura 16 – Telas de fluxo de pedidos.
Fonte: Autoria própria (2025).

Por fim, a Sprint 6 representou um avanço significativo consolidando as principais funcionalidades do sistema, garantindo maior estabilidade e segurança nas operações de pedido e pagamento. Essas melhorias aproximaram o projeto de um estado mais completo e funcional, atendendo aos objetivos estabelecidos para o MVP.

6.8 Sprint 7

A Sprint 7 teve como foco aprimorar a comunicação entre o sistema e o cliente, garantindo maior transparência e confiança durante o processo de entrega. Nesta etapa, foi desenvolvida HU13¹⁴ que consiste na funcionalidade responsável por enviar atualizações sobre o andamento do pedido, permitindo que o usuário acompanhe cada etapa da entrega.

Para possibilitar esse acompanhamento, foi desenvolvida também uma nova área para o fornecedor, que é o responsável por atualizar os status de cada pedido. Essa *dashboard* permite que o fornecedor visualize todos os pedidos realizados, filtrando-os por diferentes estados, como, por exemplo: “pendente”, “confirmado” ou “saiu para entrega”. Dessa forma, o controle das entregas se torna mais prático e centralizado em uma única tela onde se pode visualizar todas as informações importantes sobre esses pedidos, que irá refletir diretamente para o usuário do tipo cliente da aplicação, que passa a acompanhar a progressão de seu pedido com mais clareza. A figura 17 ilustra a tela desenvolvida para essa funcionalidade.



Fonte: Autoria própria (2025).

¹⁴ HU13 — **Como** cliente, **quero** receber atualizações sobre o status do meu pedido **para** saber quando ele será entregue.

Durante o desenvolvimento desta sprint, também foram realizadas melhorias estruturais no *frontend*. Após reuniões de alinhamento com o orientador do projeto, decidiu-se padronizar as rotas da aplicação, substituindo os nomes anteriormente em inglês por nomes em português, a fim de manter consistência terminológica. Essa padronização também contribui para tornar a navegação mais adequada ao público-alvo, composto por clientes e fornecedores de pequenas cidades que podem ter menor familiaridade com termos em outras línguas.

Embora o sistema já atualize corretamente o status dos pedidos, as notificações automáticas em tempo real — como *push notifications* ou atualizações via WebSocket — foram direcionadas para implementações futuras. No momento, o cliente precisa atualizar manualmente a página para visualizar a mudança de status, o que não interfere no funcionamento atual, mas representa uma possibilidade de evolução para versões posteriores da aplicação.

No *backend*, foi criado um *enum* para representar os diferentes status de um pedido, com o objetivo de padronizar esses valores e reduzir a possibilidade de inconsistências durante o processamento. O Código apresentado na listagem 4 demonstra a estrutura definida para essa funcionalidade.

Listagem 4 – Enum de status de pedido.

```

1 public enum OrderStatusEnum {
2     PENDING("Pendente"),
3     CONFIRMED("Confirmado"),
4     IN_PREPARATION("Em preparacao"),
5     OUT_FOR_DELIVERY("Saiu para entrega"),
6     DELIVERED("Entregue"),
7     CANCELED("Cancelado");
8
9     private final String status;
10
11     OrderStatusEnum(String status) {
12         this.status = status;
13     }
14
15     public String getStatus() {
16         return this.status;
17     }
18 }

```

Além disso, o *backend* foi configurado para que todo novo pedido seja criado automaticamente com o status “*Pendente*”, garantindo maior integridade e evitando erros de processamento. Essa abordagem faz com que o *frontend* apenas realize a leitura do status, sem a necessidade de enviar esse dado na criação do pedido, tornando a comunicação entre as camadas mais seguras.

Por fim, esta *sprint* consolidou a integração entre o painel do fornecedor e a experiência do cliente, aprimorando a visibilidade do ciclo total da entrega. As telas resultantes desse

desenvolvimento demonstram a evolução do sistema tanto no aspecto funcional quanto visual, representando um avanço importante.

6.9 Sprint 8

A Sprint 8 teve como principal objetivo o desenvolvimento da história HU14¹⁵, dando sequência às melhorias implementadas na Sprint 7. Esta fase buscou aprimorar a experiência do cliente por meio da criação de uma nova tela de acompanhamento de pedidos. Além do avanço funcional no *frontend*, esta *sprint* também contemplou melhorias no processo de *deploy* da aplicação, com a introdução de um fluxo automatizado de publicação na AWS, o que será detalhado posteriormente.

Nesta etapa, foi desenvolvida uma nova tela dedicada à exibição da lista dos pedidos realizados pelo cliente, permitindo que o usuário acompanhe a mudança de status de cada entrega. Essa tela foi projetada para oferecer uma visão clara e organizada, exibindo informações como a data do pedido, fornecedor responsável, valor total e a situação atual, conforme os dados atualizados pelo fornecedor.

A criação dessa interface complementa diretamente a funcionalidade introduzida na Sprint 7, que tratava da atualização do status dos pedidos (HU13). Agora, além de receber as respostas sobre as alterações do pedido, o cliente pode visualizar em um só lugar todos os pedidos já realizados com suas informações mais relevantes, reduzindo assim a necessidade de contato direto com o fornecedor, seja por outros meios como WhatsApp ou outra forma de mensagem direta, reforçando uma transparência e trazendo mais agilidade no processo de entrega. Embora o sistema ainda não utilize notificações automáticas em tempo real, o cliente pode visualizar o progresso dos pedidos ao atualizar a página, o que mantém a coerência com a arquitetura atual do projeto e assegura o correto funcionamento da aplicação. A Figura 18 ilustram a tela criada para essa *sprint*.

¹⁵ **HU14** Como cliente, quero visualizar uma lista com meus pedidos para **acompanhar as atualizações de status de cada um deles**.

Figura 18 – Tela de pedidos realizados pelo cliente

H2Gás

[Início](#) [Produtos](#) [Pedidos](#) [Cidades](#) [Fornecedores](#)

Meus Pedidos

Acompanhe seus pedidos de forma simples e rápida.

[+ Fazer novo pedido](#)

TodosPendentesConfirmadosEm preparaçãoSalu para entregaEntreguesCancelados

PEDIDO	FORNECEDOR	PRODUTO	ENDEREÇO DE ENTREGA	VALOR	STATUS
#01 08/10/2025 18:48	Admin	Produtos (2)	rua teste 233, 123 - bairro do testado	R\$ 172,00	Pendente
#02 08/10/2025 18:40	Admin	Produtos (0)	rua teste 233, 123 - bairro do testado		Cancelado
#03 08/10/2025 18:39	Admin	Produtos (0)	rua teste 233, 123 - bairro do testado		Cancelado
#04 08/10/2025 18:34	Admin	Produtos (2)	rua teste 233, 123 - bairro do testado	R\$ 172,00	Entregue
#05 08/10/2025 18:32	teste novo	Produtos (0)	rua teste 233, 123 - bairro do testado	R\$ 222,00	Pendente

[< Anterior](#) [1](#) [2](#) [Próxima >](#)

H2Gás

Sobre Contato

© 2025 H2Gás. Todos os direitos reservados.

Privacidade

Fonte: Autoria própria (2025).

Outro ponto relevante desta *sprint* foi a implementação de um processo de *deploy* automatizado na AWS, utilizando o GitHub Actions em conjunto com contêineres Docker. Até então, o *deploy* do sistema era realizado de forma manual, exigindo a execução local dos comandos do *Docker Compose*, onde sempre era necessário criar novas imagens *Docker*, o que demandava um tempo considerável. Com a nova configuração, o processo passou a ser totalmente automático, sendo acionado a partir da geração de novas *tags* no repositório GitHub. A cada nova versão criada, o *pipeline* executa a construção da imagem Docker, envia para o Docker Hub e baixa a imagem criada da nova versão no ambiente de produção na nuvem. Essa mudança trouxe maior agilidade, segurança e padronização às entregas, além de reduzir significativamente a possibilidade de erros humanos durante o processo de implantação.

O *workflow* configurado no *GitHub Actions* segue uma estrutura semelhante à exemplificada na Listagem 5.

Listagem 5 – Exemplo simplificado de workflow para deploy automatizado na AWS.

```

1  name: Deploy Automático
2
3  on:
4    push:
5      tags:
6        - 'v*.*.*' # Executa quando uma nova tag de versão é criada
7
8  jobs:
9    build-and-deploy:
10     runs-on: ubuntu-latest
11
12     steps:
13       - name: Checkout do código
14         uses: actions/checkout@v4
15
16       - name: Login no Docker Hub
17         uses: docker/login-action@v2
18         with:
19           username: ${ secrets.DOCKER_USERNAME }
20           password: ${ secrets.DOCKER_PASSWORD }
21
22       - name: Build e Push das imagens
23         run: |
24           docker build -t pabloc5/h2gas-backend:${ github.ref_name
25             ↪ } ./backend
26           docker push pabloc5/h2gas-backend:${ github.ref_name }
27           docker build -t pabloc5/h2gas-frontend:${ github.ref_name
28             ↪ } ./frontend
29           docker push pabloc5/h2gas-frontend:${ github.ref_name }
30
31       - name: Deploy na AWS
32         uses: appleboy/ssh-action@v0.1.10
33         with:
34           host: ${ secrets.AWS_HOST }
35           username: ${ secrets.AWS_USER }
36           key: ${ secrets.AWS_SSH_KEY }
37           script: |
38             TAG=${ github.ref_name }
39             cd /home/ubuntu/h2gas
40             sed -i "s|backend:.*|backend:$TAG|"
41             ↪ docker-compose.prod.yml
42             sed -i "s|frontend:.*|frontend:$TAG|"
43             ↪ docker-compose.prod.yml
44             docker compose -f docker-compose.prod.yml pull
45             docker compose -f docker-compose.prod.yml up -d

```

Esse fluxo garante que a cada nova versão marcada no repositório, o sistema seja automaticamente empacotado e publicado, reforçando a adoção de práticas de CI/CD. Com isso, o processo de atualização passou a ser mais previsível e alinhado às boas práticas de DevOps.

Por fim, esta sprint representou um avanço não apenas nas funcionalidades voltadas ao cliente, mas também na maturidade técnica e operacional do projeto. A tela de listagem de pedidos, somada à automação do deploy, reforça a qualidade, escalabilidade e a sustentabilidade do sistema a longo prazo.

6.10 Sprint 9

A Sprint 9 representou a etapa final do desenvolvimento do sistema, sendo dedicada à consolidação das funcionalidades e ajustes identificados nas reuniões de revisão das *sprints* anteriores. O principal desenvolvimento desta fase foi a implementação da história HU15¹⁶, que introduziu uma nova tela de configurações voltada aos fornecedores. Essa funcionalidade permite que o usuário do tipo fornecedor defina o estoque mínimo de produtos e os horários de funcionamento da sua loja, contribuindo para um controle eficiente das operações diárias e evitando indisponibilidades inesperadas de seus produtos. Além disso, os valores configurados de estoque mínimo passaram a ser refletidos diretamente na tela de produtos cadastrados, permitindo ao fornecedor visualizar rapidamente, por meio de indicadores nos próprios cards, quais produtos estão com níveis de estoque baixos.

Durante essa *sprint*, também foram realizadas mudanças estruturais no processo de cadastro de fornecedores. Anteriormente, era necessário criar separadamente o usuário e o fornecedor associado a ele. Após discussões e revisões com o orientador, essa abordagem foi simplificada. Agora, ao registrar um usuário do tipo fornecedor, a respectiva loja é criada automaticamente pelo *backend*, deixando então esse fornecedor em estado de aguardando análise e aprovação do administrador, eliminando assim etapas manuais e reduzindo potenciais inconsistências. Essa alteração refletiu uma importante evolução no fluxo de cadastro e na experiência de uso do sistema.

Por se tratar da última *sprint* do ciclo de desenvolvimento, também foram realizados diversos testes manuais e ajustes finais, com o objetivo de validar o funcionamento integrado de todas as funcionalidades implementadas até então. Esses testes permitiram emular o processo que o cliente fará ao utilizar o sistema e garantir a consistência e preparar a aplicação para sua entrega final. Além disso, foram revisados aspectos de infraestrutura, como o ambiente hospedado na AWS, assegurando a estabilidade e o correto funcionamento do sistema em produção.

¹⁶ **HU15** Como fornecedor, quero configurar o estoque mínimo e o horário de funcionamento da minha loja para **controlar a disponibilidade dos produtos e gerenciar melhor as entregas**.

6.11 Considerações sobre o desenvolvimento

O desenvolvimento do sistema foi conduzido de forma iterativa e incremental, seguindo o planejamento definido nas *sprints*. Ao longo das etapas, foram implementadas as principais funcionalidades propostas, passando por fases de testes, revisões e ajustes sugeridos durante as reuniões com o orientador. Esta seção apresenta as considerações finais sobre o processo de desenvolvimento, destacando as melhorias alcançadas, as decisões técnicas mais relevantes e a consolidação das entregas que resultaram na versão final do sistema.

6.11.1 Desafios e Adaptações

Durante o desenvolvimento do sistema, alguns desafios e adaptações foram necessários para aprimorar a estrutura e a escalabilidade do projeto. Um dos principais ajustes ocorreu no banco de dados, com a divisão da antiga tabela *products* em três novas entidades: *product_item*, *product_type* e *category_products*. Essa mudança teve como objetivo melhorar a organização das informações e permitir uma evolução mais flexível da aplicação em futuras versões.

No *frontend*, a utilização do *framework* Angular demonstrou-se bastante vantajosa, principalmente pela facilidade de criação e reutilização de componentes, o que acelerou a implementação das telas e contribuiu para uma melhor manutenção do código e também a padronização da interface.

No lado do servidor, o uso do *Spring Framework* trouxe benefícios significativos na construção da API, permitindo uma implementação mais organizada e ágil. Além disso, o suporte nativo a boas práticas, como injeção de dependência e camadas bem definidas, contribuiu para a clareza e a escalabilidade do projeto. Outro ponto relevante foi a utilização do *Spring Security*, que facilitou a implementação dos mecanismos de autenticação e autorização, garantindo maior segurança no acesso das rotas e funcionalidades do sistema e reforçando a estrutura de controle de usuários e permissões.

6.11.2 Aprendizado Técnico

O processo de desenvolvimento do sistema proporcionou uma série de aprendizados importantes, tanto ao nível técnico quanto prático. No *backend*, a implementação de funcionalidades como o processo de redefinição de senhas e o envio de e-mails utilizando o *Spring Boot* ampliou o domínio sobre o funcionamento de serviços assíncronos e integração com provedores externos. A utilização do *Spring Security* também foi uma experiência enriquecedora, permitindo compreender de forma mais aprofundada a estrutura de autenticação e autorização que contribui para uma aplicação mais segura e confiável.

Outro ponto relevante foi o contato com aspectos de design e usabilidade, que exigiram uma maior atenção à experiência do usuário e ao layout das interfaces. Essa etapa permitiu aprimorar conhecimentos sobre organização visual e boas práticas de apresentação.

Por fim, o desenvolvimento e execução de testes automatizados com o *JUnit*, em conjunto com o banco de dados em memória *H2*, contribuíram significativamente para o entendimento sobre validação de funcionalidades e garantia de qualidade do código. Esses testes auxiliaram na detecção de possíveis falhas ainda durante a fase de desenvolvimento.

6.11.3 Deploy

A etapa de implantação do sistema foi realizada utilizando a infraestrutura da AWS, aproveitando os recursos do plano gratuito oferecido pela plataforma. Isso possibilitou garantir a hospedagem da aplicação e a disponibilização do sistema de maneira estável.

Todo o ambiente foi configurado em contêineres *Docker*, o que permitiu padronizar o processo de execução e facilitar a portabilidade entre diferentes ambientes. A utilização de contêineres também simplificou a manutenção e a replicação do ambiente de desenvolvimento.

Além disso, foi configurado um fluxo automatizado de integração e entrega contínua CI/CD por meio do *GitHub Actions*. Esse processo automatiza etapas como build, testes e deploy, permitindo que novas atualizações sejam publicadas de forma mais ágil e controlada. Essa prática contribuiu para a eficiência do desenvolvimento, além de aproximar o projeto de metodologias que são hoje amplamente utilizadas no mercado profissional. A configuração desse *pipeline* também representou um importante aprendizado, possibilitando reforçar conhecimentos sobre automação de processos e integração entre diferentes etapas do ciclo de desenvolvimento, áreas que até então apresentavam menor familiaridade e que se mostraram essenciais.

7 CONCLUSÃO

O sistema desenvolvido representa um protótipo funcional voltado à gestão de pedidos de gás e água, idealizado para validar a viabilidade técnica e operacional de uma aplicação voltada a esse segmento. O projeto surgiu a partir da identificação de uma lacuna no atendimento digital desse tipo de serviço em cidades pequenas, onde o contato entre clientes e distribuidores ainda ocorre, predominantemente, de forma presencial ou por meio de aplicativos de mensagens, como o WhatsApp. Dessa forma, a aplicação foi concebida como um MVP, com o propósito de testar as principais funcionalidades e interações entre os usuários, avaliando sua estrutura, usabilidade e potencial de expansão em versões futuras.

Com o desenvolvimento do *MVP*, foi possível atingir os principais objetivos definidos no início do projeto. Entre os resultados alcançados, destacam-se:

- O desenvolvimento de uma **plataforma digital funcional**, capaz de substituir métodos tradicionais de solicitação de pedidos, como o contato telefônico;
- A implementação de mecanismos que proporcionam **maior agilidade e organização** no processo de compra e entrega de produtos;
- A disponibilização de recursos que permitem a **comparação entre diferentes fornecedores**, tornando o processo de escolha mais transparente e informativo;
- A criação de um ambiente de **gerenciamento simplificado de vendas e entregas** para os distribuidores.

Embora o sistema tenha sido concluído em sua versão inicial, ele ainda não foi submetido a testes com usuários reais. Assim, uma etapa essencial para trabalhos futuros consiste na realização de testes de usabilidade e validação com distribuidores e consumidores, a fim de coletar feedbacks, identificar pontos de melhoria e verificar a adequação da solução no contexto prático. Esses procedimentos possibilitarão ajustes no sistema e a expansão gradual de suas funcionalidades, contribuindo para seu uso como ferramenta de apoio na gestão de pedidos locais.

Considerando o escopo proposto para o MVP e o tempo disponível para o desenvolvimento, algumas funcionalidades previstas inicialmente não foram implementadas nesta versão. Essas melhorias e extensões estão detalhadas como sugestões para trabalhos futuros na Seção 7.1, servindo de base para a continuidade e evolução do projeto em etapas posteriores.

7.1 Trabalhos Futuros

Finalizado o desenvolvimento do projeto proposto como um MVP, a aplicação H2Gás atingiu seu objetivo principal ao contemplar as funcionalidades essenciais para o uso prático

pelos clientes e distribuidores. Essa versão inicial possibilitou validar a proposta e demonstrar o potencial do sistema em ambientes reais. No entanto, durante o processo de desenvolvimento e nos testes realizados, foram identificadas diversas oportunidades de aprimoramento, tanto em termos técnicos quanto funcionais. As funcionalidades e melhorias que não foram contempladas nesta etapa estão apresentadas na Tabela 2, a qual reúne as sugestões de evolução para versões futuras do sistema.

Entre as principais propostas de aprimoramento, destaca-se a implementação de notificações em tempo real (*push notifications*), que permitirão o envio instantâneo de atualizações sobre o status dos pedidos, como confirmação, preparo e entrega. Outra melhoria prevista é a inclusão de um sistema de avaliação e feedback, possibilitando que os clientes avaliem o atendimento e os fornecedores acompanhem o desempenho de seus serviços, promovendo um ciclo contínuo de aprimoramento e confiança.

Por fim, também se planeja a adição de um novo tipo de usuário, o *Entregador*, cuja implementação foi adiada devido ao escopo do MVP. Esse perfil permitirá gerenciar de forma mais completa o processo de entrega, desde a aceitação dos pedidos até a confirmação final, além de possibilitar comunicação direta com o cliente. Todas essas evoluções estão descritas na Tabela 2, que apresenta as histórias de usuário correspondentes às funcionalidades planejadas para futuras versões do sistema.

Tabela 2 – Funcionalidades previstas para versões futuras do sistema

ID	Como	Quero	Para
HU16	Administrador	coletar feedback dos clientes após a entrega	possibilitar melhorias contínuas no serviço
HU18	Administrador	gerenciar programas de fidelidade e promoções	incentivar a retenção e o engajamento dos clientes
HU19	Fornecedor	acessar relatórios de vendas e estatísticas de entrega	tomar decisões estratégicas e aprimorar o serviço
HU20	Fornecedor	interagir com o cliente diretamente no aplicativo	esclarecer dúvidas durante o processo de pedido
HU21	Cliente	agendar a entrega para um horário específico	obter maior flexibilidade e conveniência
HU23	Cliente	avaliar o serviço e indicar amigos para o aplicativo	promover o uso da plataforma e receber benefícios
HU24	Entregador	visualizar as avaliações recebidas dos clientes	obter feedback e melhorar continuamente o atendimento
HU25	Entregador	comunicar-se diretamente com o cliente	confirmar o endereço ou esclarecer dúvidas sobre a entrega
HU26	Administrador	implementar elementos de gamificação, como badges e recompensas	aumentar o engajamento dos usuários
HU27	Fornecedor	gerenciar o estoque em tempo real	evitar pedidos de produtos indisponíveis e otimizar o atendimento
HU29	Cliente	cadastrar e gerenciar múltiplos endereços	facilitar pedidos para diferentes locais, como casa e trabalho
HU30	Entregador	receber notificações em tempo real (push notifications) sobre novos pedidos disponíveis	gerenciar de forma mais eficaz as entregas e aceitar rapidamente novos pedidos
HU31	Cliente	realizar pedidos por comando de voz através de assistentes virtuais	proporcionar mais comodidade e acessibilidade ao usuário
HU32	Cliente	receber notificações em tempo real (push notifications) sobre o status do pedido	acompanhar atualizações importantes, como confirmação, saída para entrega e conclusão

Fonte: Autoria própria (2025).

REFERÊNCIAS

- Aiqfome. **Como apoiamos a economia local das cidades do interior**. 2024. Frigideira Aiqfome. Disponível em: <https://frigideira.aiqfome.com/economia-local/>. Acesso em: 12 nov. 2024.
- Angular Developers. **Angular Documentation**. 2025. Disponível em: <https://angular.dev/overview>. Acesso em: 1 fev. 2025.
- Atlassian. **Saiba tudo sobre o Gitflow Workflow**. 2025. Disponível em: <https://www.atlassian.com/br/git/tutorials/comparing-workflows/gitflow-workflow>. Acesso em: 5 fev. 2025.
- AWS. **Overview of Amazon Web Services**. 2025. <https://docs.aws.amazon.com/whitepapers/latest/aws-overview/introduction.html>. Acesso em: 5 nov. 2025.
- CNN. **Entenda como funciona e quando usar a metodologia Scrum**. 2023. Acesso em: 23 fev. 2025. Disponível em: <https://www.cnnbrasil.com.br/tecnologia/scrum/>.
- DRUMOND, C. **O que é scrum e como começar**. 2025. Atlassian. Acesso em: 6 fev. 2025. Disponível em: <https://www.atlassian.com/br/agile/scrum>.
- Eagle, Alex et al. **Monorepo.tools: Everything you need to know about monorepos, and the tools to build them**. 2025. Disponível em: <https://monorepo.tools/#what-is-a-monorepo>. Acesso em: 7 fev. 2025.
- GILL, N. S. **Kanban: conceito, como funciona, vantagens e implementação**. 2024. Acesso em: 23 fev. 2025. Disponível em: <https://www.xenonstack.com/blog/test-driven-development-java>.
- GitHub. **Sobre o Git: Saiba mais sobre o sistema de controle de versões, Git, e como ele funciona com GitHub**. 2025. Disponível em: <https://docs.github.com/pt/get-started/using-git/about-git>. Acesso em: 5 fev. 2025.
- Github. **Sobre Projects**. 2025. Disponível em: <https://docs.github.com/pt/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects>. Acesso em: 6 fev. 2025.
- IBM. **O que é Java Spring Boot?** 2025. Disponível em: <https://www.ibm.com/br-pt/topics/java-spring-boot>. Acesso em: 6 fev. 2025.
- iFood. **O que é o iFood? Conheça a história e a operação da empresa**. 2023. Institucional iFood. Disponível em: <https://institucional.ifood.com.br/noticias/o-que-e-o-ifood/>. Acesso em: 12 nov. 2024.
- iFood. **Taxas iFood: tire suas dúvidas sobre o assunto**. 2024. IFood Blog Parceiros. Disponível em: <https://blog-parceiros.ifood.com.br/taxas-ifood/>. Acesso em: 12 nov. 2024.
- JÚNIOR, F. Delivery transformou tendência em necessidade e continua em crescimento. **Jornal da USP**, mar. 2021. Disponível em: <https://jornal.usp.br/atualidades/delivery-transformou-tendencia-em-necessidade-e-continua-em-crescimento/>. Acesso em: 29 oct. 2024.
- MADUREIRA, D. **Como apps de entrega estão levando pequenos restaurantes à falência**. 2020. De São Paulo para a BBC News Brasil. Disponível em: <https://www.bbc.com/portuguese/geral-51272233>. Acesso em: 20 oct. 2024.

Oracle. **O que é o MySQL?** 2025. Disponível em: <https://www.oracle.com/br/mysql/what-is-mysql/>. Acesso em: 7 fev. 2025.

Rappi. **Taxas do Rappi para Restaurantes.** 2024. Rappi Merchants. Disponível em: <https://merchants.rappi.com/pt-br/taxas-rappi-para-restaurantes>. Acesso em: 12 nov. 2024.

Redhat. **Docker: desenvolvimento de aplicações em containers.** 2023. Redhat. Disponível em: <https://www.redhat.com/pt-br/topics/containers/what-is-docker>. Acesso em: 7 fev. 2025.

Saadeghi, Pouya. **What is daisyUI?** 2025. Disponível em: <https://daisyui.com/blog/what-is-daisyui/>. Acesso em: 3 fev. 2025.

Spring. **Spring Framework Reference Documentation.** [S.l.], 2024. Disponível em: <https://docs.spring.io/spring-framework/docs/3.2.x/spring-framework-reference/html/mvc.html>. Acesso em: 7 fev. 2025.

STEINFELD, G. **5 steps of test-driven development Learn about test-driven development and see it in action.** 2024. Acesso em: 23 fev. 2025. Disponível em: <https://developer.ibm.com/articles/5-steps-of-test-driven-development/>.

TOTVS. **Kanban: conceito, como funciona, vantagens e implementação.** 2023. Acesso em: 23 fev. 2025. Disponível em: <https://www.totvs.com/blog/negocios/kanban/>.

UFRJ, F. C. **Boas práticas de Programação e a importância dos testes.** 2025. Acesso em: 23 fev. 2025. Disponível em: <https://fluxoconsultoria.poli.ufrj.br/blog/boas-praticas-desenvolvimento-software/>.

Ultragaz. **Peça seu gás sem sair de casa com a qualidade e rapidez Ultragaz.** 2020. Ultragaz. Disponível em: <https://www.ultragaz.com.br/app/>. Acesso em: 12 nov. 2024.

World Health Organization. **Household air pollution.** 2024. World Health Organization. Disponível em: <https://www.who.int/news-room/fact-sheets/detail/household-air-pollution-and-health>. Acesso em: 06 nov. 2024.

APÊNDICE A – Funcionalidades Classificadas como Should Have, Could Have e Won't Have (MoSCoW)

A.1 Funcionalidades Should Have (Deveria Ter)

Essas funcionalidades são importantes para enriquecer a experiência do usuário e facilitar a operação do sistema, mas o aplicativo ainda pode cumprir seu propósito sem elas.

- **Administrador**

- **Coleta de Feedback de Clientes:** Ferramenta para coletar feedback dos clientes após a entrega, possibilitando melhorias contínuas no serviço.
- **Configuração de Segurança:** Implementação de protocolos de segurança para proteção de dados, garantindo a privacidade das informações de clientes, fornecedores e entregadores.
- **Gerenciamento de Programas de Fidelidade e Promoções:** O administrador poderá configurar programas de fidelidade e promoções para incentivar a retenção e engajamento dos clientes.

- **Fornecedor**

- **Acesso a Relatórios e Estatísticas:** Relatórios de vendas e estatísticas de entrega para permitir que o fornecedor tome decisões estratégicas e melhore seu serviço.
- **Interação com o Cliente:** Ferramenta de comunicação para esclarecer dúvidas diretamente com o cliente durante o processo de pedido.

- **Cliente**

- **Entrega Agendada:** Opção de agendar a entrega para um horário específico, oferecendo flexibilidade e conveniência ao cliente.
- **Histórico de Compras:** Acesso ao histórico de pedidos para facilitar compras futuras e visualizar informações sobre transações passadas.
- **Avaliação do Serviço e Indicação de Amigos:** O cliente poderá avaliar o serviço recebido e convidar amigos para utilizar o aplicativo, promovendo o uso da plataforma e recebendo benefícios.

- **Entregador**

- **Visualização de Avaliações de Clientes:** O entregador terá acesso a avaliações recebidas, permitindo feedback sobre seu serviço e incentivando melhorias contínuas.
- **Comunicação Direta com o Cliente:** Ferramenta de comunicação para permitir que o entregador contate o cliente diretamente para confirmar o endereço ou esclarecer dúvidas sobre a entrega.

A.2 Funcionalidades Could Have (Poderia Ter)

As funcionalidades Could Have são recursos desejáveis que podem enriquecer ainda mais o aplicativo, mas não são essenciais para o funcionamento inicial.

- **Administrador**

- **Elementos de Gamificação:** Implementação de elementos de gamificação, como badges e recompensas, para aumentar o engajamento dos usuários.

- **Fornecedor**

- **Gerenciamento de Estoque em Tempo Real:** O fornecedor poderá gerenciar seu estoque no aplicativo para evitar pedidos de produtos indisponíveis e otimizar a satisfação do cliente.

- **Cliente**

- **Pedidos Offline:** A possibilidade de adicionar produtos ao carrinho e salvar pedidos offline, permitindo o uso do app mesmo em áreas com baixa conectividade.
- **Cadastro de Múltiplos Endereços:** Opção para adicionar e gerenciar múltiplos endereços, como casa e trabalho, facilitando pedidos para diferentes locais.

- **Entregador**

- **Notificações de Novos Pedidos:** Notificações automáticas sobre novos pedidos disponíveis para entrega, permitindo uma gestão mais eficaz das entregas.

A.3 Funcionalidades Won't Have This Time (Não Terá Desta Vez)

Essas funcionalidades foram identificadas como interessantes para futuras versões do sistema, mas foram deixadas de lado nesta fase inicial do projeto devido a limitações de tempo ou recursos.

- **Integração com Assistentes Virtuais:** A possibilidade de integração com assistentes virtuais como Alexa e Google Assistant para pedidos por comando de voz pode ser implementada em versões futuras, proporcionando ainda mais comodidade ao cliente.